

ESP32 Audio FX

Fonctionnement détaillé des effets audio

Traitement numérique du signal sur microcontrôleur

Ce document explique, effet par effet, ce que fait chaque traitement du processeur, les mathématiques qui le décrivent et la façon dont il est implémenté dans le firmware. **Toutes les courbes sont calculées en exécutant le vrai code C++ des effets** (compilé sur PC) : elles montrent ce que l'ESP32 fait réellement, pas un modèle idéalisé.

Bluetooth A2DP → 15 effets + Master → DAC PCM5102

Freenove ESP32-WROVER-E · écran ST7735 · 17 octobre 2026

Table des matières

1 Introduction **4**
- 1.1 Ce que fait le processeur . . . 4
- 1.2 Comment lire ce document . . . 4

2 Notions fondamentales **6**
- 2.1 Signal numérique et échantillonnage . . . 6
  - 2.1.1 Formats numériques . . . 6
  - 2.1.2 Traitement par blocs . . . 6
- 2.2 Décibels . . . 6
- 2.3 Systèmes linéaires et transformée en z . . . 7
  - 2.3.1 Équation aux différences . . . 7
  - 2.3.2 Fonction de transfert . . . 7
  - 2.3.3 Réponse en fréquence . . . 7
- 2.4 Les briques de base . . . 7
  - 2.4.1 Filtre à un pôle : lissage et tonalité . . . 7
  - 2.4.2 Le biquad . . . 8
  - 2.4.3 Ligne à retard . . . 8
  - 2.4.4 Oscillateurs basse fréquence (LFO) . . . 9
  - 2.4.5 Saturation douce . . . 10
  - 2.4.6 Stéréo : Mid/Side et loi de panoramique . . . 10

3 Architecture du traitement **11**
- 3.1 Deux cœurs, trois rôles . . . 11
- 3.2 Chemin des échantillons . . . 11
  - 3.2.1 Le tampon anti-gigue . . . 11
  - 3.2.2 Sortie I2S et cadencement . . . 12
  - 3.2.3 Latence . . . 12
  - 3.2.4 Mesure de la charge . . . 12
- 3.3 La classe parent `Effect` . . . 12
  - 3.3.1 Paramètres . . . 13
  - 3.3.2 Activation sans clic . . . 13
  - 3.3.3 Affichage . . . 14
  - 3.3.4 Ajouter un effet . . . 14
- 3.4 Ordre de la chaîne . . . 15

4 Filtres et égalisation **16**
- 4.1 La classe intermédiaire `FilterEffect` . . . 16
  - 4.1.1 Paramètres communs . . . 16
  - 4.1.2 Modèle : du filtre analogique au biquad . . . 16
  - 4.1.3 Implémentation . . . 17
- 4.2 Low Pass (passe-bas) . . . 18
- 4.3 High Pass (passe-haut) . . . 18
- 4.4 Band Pass (passe-bande) . . . 19
- 4.5 EQ 3 bandes . . . 19
  - 4.5.1 Paramètres . . . 19
  - 4.5.2 Modèle . . . 20
  - 4.5.3 Implémentation et visualisation . . . 20

5 Dynamique et distorsion **21**
5.1 Compressor . . . 21
5.1.1 Paramètres . . . 21
5.1.2 Schéma-bloc . . . 21
5.1.3 Courbe statique et genou doux . . . 21
5.1.4 Attaque et relâche . . . 22
5.1.5 Implémentation . . . 23
5.2 Distortion . . . 23
5.2.1 Paramètres . . . 23
5.2.2 Chaîne de traitement . . . 23
5.2.3 Les quatre courbes . . . 24
5.2.4 Pourquoi des harmoniques ? . . . 24
5.2.5 Étages auxiliaires . . . 25
5.3 Bitcrusher . . . 25
5.3.1 Paramètres . . . 26
5.3.2 Quantification . . . 26
5.3.3 Réduction d'échantillonnage . . . 26
5.4 Ring Mod (modulateur en anneau) . . . 27
5.4.1 Paramètres . . . 27
5.4.2 Modèle . . . 27
5.4.3 Implémentation . . . 28

6 Effets de modulation **29**
6.1 Tremolo . . . 29
6.1.1 Paramètres . . . 29
6.1.2 Modèle . . . 29
6.2 AutoPan . . . 30
6.2.1 Paramètres . . . 30
6.2.2 Modèle : panoramique à puissance constante . . . 30
6.3 Phaser . . . 31
6.3.1 Paramètres . . . 31
6.3.2 Le filtre passe-tout . . . 31
6.3.3 Les creux . . . 31
6.3.4 Implémentation . . . 32
6.4 Flanger . . . 33
6.4.1 Paramètres . . . 33
6.4.2 Modèle . . . 33
6.4.3 Implémentation . . . 34
6.5 Chorus . . . 34
6.5.1 Paramètres . . . 34
6.5.2 Modèle . . . 34
6.5.3 D'où vient le désaccord ? L'effet Doppler . . . 35

7 Effets temporels **36**
7.1 Delay . . . 36
7.1.1 Paramètres . . . 36
7.1.2 Schéma-bloc . . . 36
7.1.3 Modèle . . . 36
7.1.4 Détails d'implémentation . . . 37
7.2 Reverb . . . 38
7.2.1 Paramètres . . . 38
7.2.2 L'algorithme Freeverb . . . 38
7.2.3 Le peigne amorti . . . 39

7.2.4 Le diffuseur . . . 40
7.2.5 Mélange stéréo . . . 40
7.2.6 Implémentation . . . 40

8 Sortie et analyse **42**
8.1 Master . . . 42
8.1.1 Paramètres . . . 42
8.1.2 Chaîne . . . 42
8.1.3 Le limiteur . . . 42
8.2 L'analyseur de spectre . . . 43
8.2.1 Capture . . . 43
8.2.2 Fenêtre de Hann . . . 43
8.2.3 La FFT radix-2 . . . 44
8.2.4 Des raies aux bandes affichées . . . 44

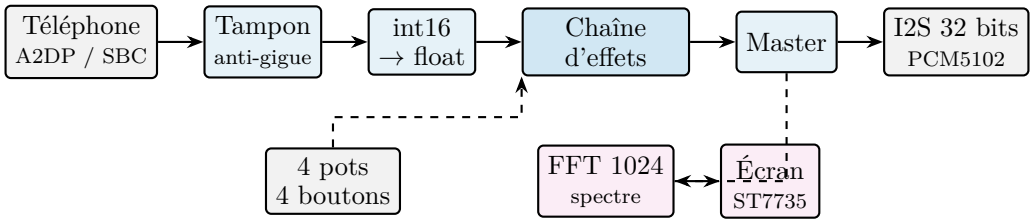
A Récapitulatif des effets **46**
A.1 Paramètres . . . 46
A.2 Coût de calcul et mémoire . . . 46
A.3 Références . . . 47

CHAPITRE 1

Introduction

1.1 Ce que fait le processeur

Le processeur reçoit de la musique depuis un téléphone en **Bluetooth** (profil A2DP), la fait passer dans une **chaîne d'effets** réglable puis l'envoie à un **convertisseur numérique-analogique** (DAC PCM5102) relié à un ampli ou à un casque. Un écran couleur affiche les menus, la page de réglage de chaque effet et le **spectre** du signal de sortie, calculé en temps réel par transformée de Fourier.



Le processeur propose **quinze effets**, chacun programmé comme une classe C++ qui hérite d'une classe parent commune `Effect`, plus un étage **Master** toujours actif :

Famille	Effets
Filtres et égalisation	Low Pass, High Pass, Band Pass, EQ 3 bandes
Dynamique et distorsion	Compressor, Distortion, Bitcrusher, Ring Mod
Modulation	Tremolo, AutoPan, Phaser, Flanger, Chorus
Effets temporels	Delay, Reverb
Sortie	Master (volume, balance, largeur stéréo, limiteur)

1.2 Comment lire ce document

Le chapitre 2 rassemble les notions de traitement du signal utilisées partout ensuite (échantillonnage, décibels, transformée en z, filtres, oscillateurs…). Le chapitre 3 décrit l'organisation du logiciel : tâches temps réel, tampon d'entrée, classe `Effect`. Les chapitres suivants détaillent les effets, famille par famille, toujours sur le même plan :

1. **Principe** : ce que l'on entend, et pourquoi.
2. **Modèle mathématique** : équations aux différences, fonctions de transfert.
3. **Schéma-bloc** : le chemin du signal.
4. **Paramètres** : ce que règle chacun des quatre potentiomètres, plage et échelle.
5. **Implémentation** : extraits commentés du code réel.
6. **Mesures** : courbes obtenues en exécutant le vrai code.
7. **Remarques** : coût de calcul, limites, pièges.

Le dernier chapitre traite du Master et de l'analyseur de spectre. Les annexes récapitulent les paramètres et l'occupation mémoire.

> **Conventions.** Le signal est représenté par des nombres à virgule flottante dans l'intervalle $[-1, 1]$; 1 correspond à la pleine échelle du DAC (0 dBFS). On note $x[n]$ l'échantillon d'entrée

numéro n, $y[n]$ la sortie, f_s la fréquence d'échantillonnage (44 100 Hz en pratique), L et R les canaux gauche et droit.

CHAPITRE 2

Notions fondamentales

Ce chapitre présente les outils mathématiques et les « briques » de code réutilisées par tous les effets. Les briques se trouvent dans `src/dsp/`.

2.1 Signal numérique et échantillonnage

Un son est une variation de pression. Le téléphone la transmet sous forme d'une suite de nombres, les *échantillons*, mesurés $f_s = 44\,100$ fois par seconde pour chaque canal. Le théorème de Shannon-Nyquist dit qu'on peut représenter sans perte toutes les fréquences inférieures à $f_s/2 = 22\,050$ Hz, la *fréquence de Nyquist*, ce qui couvre l'audition humaine (environ 20 Hz à 20 000 Hz).

Toute fréquence supérieure à $f_s/2$ produite par un calcul (par exemple une distorsion qui crée des harmoniques) est *repliée* : elle réapparaît à la fréquence $f_s - f$. C'est le **repliement spectral** (*aliasing*), qui explique certains sons métalliques de la distorsion et du bitcrusher.

2.1.1 Formats numériques

Étape	Format	Conversion
Bluetooth (après décodage SBC)	entiers 16 bits signés, stéréo entrelacé	—
Traitement	`float` 32 bits dans $[-1, 1]$	$x = s_{16}/32768$
Sortie I2S vers le DAC	entiers 32 bits signés	$s_{32} = \lfloor x \cdot 2^{31} \rfloor$ (écrêté)

Le calcul en flottant évite de gérer les débordements et la perte de précision : le cœur de l'ESP32 possède une unité de calcul flottant simple précision. La sortie en 32 bits conserve la précision obtenue (les queues de réverbération, très faibles, ne sont pas tronquées à 16 bits).

2.1.2 Traitement par blocs

Les échantillons sont traités par **blocs de** $B = 128$ **trames** (une trame = un échantillon gauche + un droit). Un bloc dure

$$T_B = \frac{B}{f_s} = \frac{128}{44100} \approx 2{,}9\,\text{ms}.$$

Chaque effet reçoit deux tableaux `L[128]` et `R[128]` qu'il modifie *sur place*. Les calculs coûteux qui n'ont pas besoin d'être faits à chaque échantillon (coefficients de filtres, par exemple) sont faits une fois par bloc, soit $f_s/B \approx 344$ fois par seconde.

2.2 Décibels

L'oreille perçoit les niveaux de façon logarithmique. On exprime donc un rapport d'amplitude g en **décibels** :

$$G_{\text{dB}} = 20 \log_{10} g \qquad \Longleftrightarrow \qquad g = 10^{G_{\text{dB}}/20}.$$

Doubler l'amplitude ajoute ≈ 6 dB ; diviser par 10 retire 20 dB. Un niveau en **dBFS** (*full scale*) est mesuré par rapport à la pleine échelle : 0 dBFS est le maximum, tout le reste est négatif. Dans le code : `dsp::dbToGain()` et `dsp::gainToDb()` (fichier `DspUtil.h`).

2.3 Systèmes linéaires et transformée en z

2.3.1 Équation aux différences

Un filtre numérique calcule chaque sortie à partir des entrées et des sorties précédentes :

$$y[n] = \sum_{k=0}^{M} b_k\, x[n-k] \;-\; \sum_{k=1}^{N} a_k\, y[n-k].$$

Les termes b_k (directs) forment des combinaisons d'échos, les termes a_k (récursifs) réinjectent la sortie : c'est la *récursivité* qui donne la résonance.

2.3.2 Fonction de transfert

La transformée en z remplace « retarder d'un échantillon » par « multiplier par z^{-1} ». L'équation précédente devient un rapport de polynômes, la **fonction de transfert** :

$$H(z) = \frac{Y(z)}{X(z)} = \frac{b_0 + b_1 z^{-1} + \cdots + b_M z^{-M}}{1 + a_1 z^{-1} + \cdots + a_N z^{-N}}.$$

2.3.3 Réponse en fréquence

Pour connaître l'effet du filtre sur une sinusoïde de fréquence f, on évalue H sur le cercle unité, en $z = \mathrm{e}^{\mathrm{j}\omega}$ avec la pulsation normalisée

$$\omega = 2\pi \frac{f}{f_s}.$$

Le module $|H(\mathrm{e}^{\mathrm{j}\omega})|$ est le gain appliqué à cette fréquence et l'argument $\arg H(\mathrm{e}^{\mathrm{j}\omega})$ son déphasage. C'est exactement ce que calcule `BiquadCoefs::magnitudeDb()` pour tracer les courbes de réponse à l'écran : en écrivant $\mathrm{e}^{-\mathrm{j}\omega} = \cos\omega - \mathrm{j}\sin\omega$,

$$|H|^2 = \frac{(b_0 + b_1\cos\omega + b_2\cos 2\omega)^2 + (b_1\sin\omega + b_2\sin 2\omega)^2}{(1 + a_1\cos\omega + a_2\cos 2\omega)^2 + (a_1\sin\omega + a_2\sin 2\omega)^2}.$$

2.4 Les briques de base

2.4.1 Filtre à un pôle : lissage et tonalité

Le filtre récursif le plus simple est

$$y[n] = (1-a)\,x[n] + a\,y[n-1], \qquad 0 \le a < 1,$$

de fonction de transfert $H(z) = \dfrac{1-a}{1-az^{-1}}$. C'est un **passe-bas du premier ordre** (pente 6 dB par octave). Deux façons de choisir a :

— par une **fréquence de coupure** f_c : $a = \mathrm{e}^{-2\pi f_c/f_s}$;

— par une **constante de temps** τ (temps pour parcourir 63 % du chemin vers la cible) : $a = \mathrm{e}^{-1/(\tau f_s)}$. C'est le cas des *lissages* de paramètres, des détecteurs d'enveloppe et des attaques/relâchements du compresseur.

Dans le code — `dsp/Biquad.h`, `dsp/DspUtil.h`

```
struct OnePoleLP {
  float a = 0, z = 0;
  void setCutoff(float fc, float fs) { a = expf(-TWO_PI_F * fc / fs); }
  inline float process(float x) { z = x + a * (z - x); return z; }  // = (1-a)x + a z
```

```
};
inline float onePoleCoef(float timeMs, float fs) { return expf(-1.f / (timeMs * 0.001f * fs)); }
```

On rencontre souvent dans le code la forme équivalente « `v += (cible - v) * k` » : c'est le même filtre avec $k = 1 - a$. Pour un coefficient k appliqué à chaque échantillon, la constante de temps vaut $\tau \approx 1/(k f_s)$.

2.4.2 Le biquad

Le **biquad** est un filtre du second ordre ($M = N = 2$) :

$$H(z) = \frac{b_0 + b_1 z^{-1} + b_2 z^{-2}}{1 + a_1 z^{-1} + a_2 z^{-2}}.$$

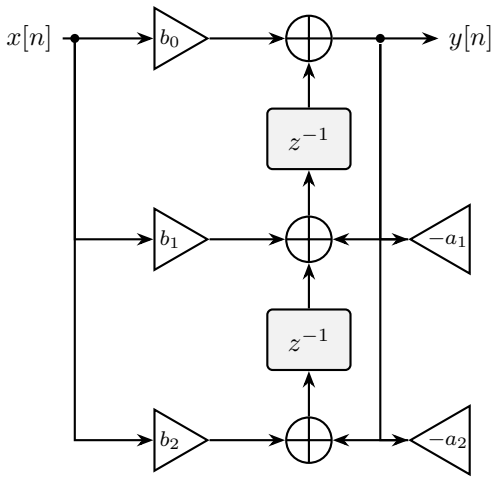
Il suffit à réaliser tous les filtres usuels (passe-bas, passe-haut, passe-bande, cloche, plateaux) en changeant seulement ses cinq coefficients. Ceux-ci sont calculés avec les formules classiques de Robert Bristow-Johnson (*Audio EQ Cookbook*), à partir de la fréquence f_0, du facteur de qualité Q et éventuellement d'un gain G :

$$\omega_0 = 2\pi \frac{f_0}{f_s}, \qquad \alpha = \frac{\sin \omega_0}{2Q}, \qquad A = 10^{G/40}.$$

Les formules propres à chaque type sont données avec les effets correspondants (chapitre 4).

Structure de calcul. On utilise la **forme directe transposée II**, qui n'a besoin que de deux variables d'état (z_1, z_2) et se comporte bien en virgule flottante :

$$\begin{aligned} y[n] &= b_0\, x[n] + z_1, \\ z_1 &\leftarrow b_1\, x[n] - a_1\, y[n] + z_2, \\ z_2 &\leftarrow b_2\, x[n] - a_2\, y[n]. \end{aligned}$$



2.4.3 Ligne à retard

Une **ligne à retard** mémorise les derniers échantillons dans un **tampon circulaire** : un index d'écriture avance d'une case à chaque échantillon et revient au début en fin de tableau. Lire D cases derrière l'index donne $x[n - D]$. Les grands tampons sont alloués dans la **PSRAM** externe de 8 Mo du module WROVER (`dsp::allocBuffer`).

Pour un retard **fractionnaire** $D = d + \mu$ (d entier, $0 \le \mu < 1$), indispensable aux effets dont le retard varie en continu (chorus, flanger, delay), on interpole linéairement :

$$x[n - D] \approx (1 - \mu)\, x[n - d] + \mu\, x[n - d - 1].$$

Dans le code — `dsp/DelayLine.h`

```
inline float readFrac(float delay) const {
  delay = clampf(delay, 1.f, (float)(size_ - 3));
  const int di = (int)delay;
  const float f = delay - di;
  const float a = read(di), b = read(di + 1);
  return a + (b - a) * f;              // interpolation lineaire
}
```

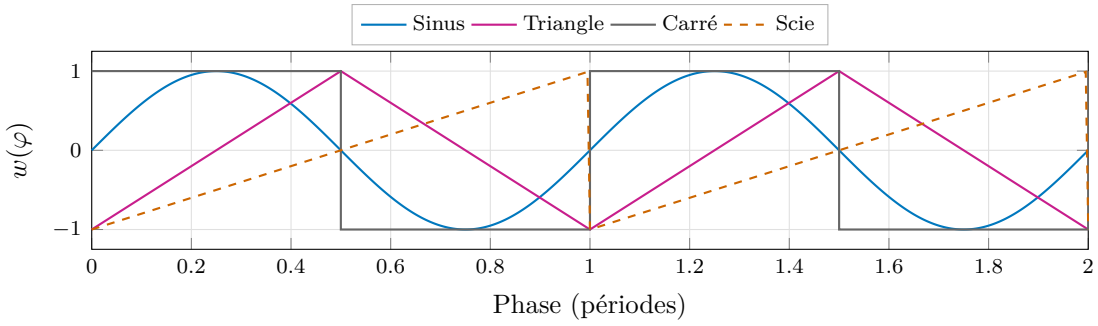
L'interpolation linéaire atténue légèrement les aigus quand $\mu \approx 0{,}5$ (c'est un petit passe-bas). Pour des effets de modulation, c'est inaudible et beaucoup moins coûteux qu'une interpolation d'ordre supérieur.

2.4.4 Oscillateurs basse fréquence (LFO)

Les effets de modulation font varier un paramètre selon une onde lente, de 0,05 Hz à 15 Hz : le **LFO** (*low frequency oscillator*).

Accumulateur de phase. La phase $\varphi \in [0, 1[$ avance de f/f_s à chaque échantillon et revient à 0 en dépassant 1. La forme d'onde est une fonction de la phase :

$$w_{\sin}(\varphi) = \sin 2\pi\varphi, \quad w_{\text{tri}}(\varphi) = \begin{cases} 4\varphi - 1 & \varphi < \frac{1}{2} \\ 3 - 4\varphi & \text{sinon} \end{cases}, \quad w_{\text{carré}}(\varphi) = \pm 1, \quad w_{\text{scie}}(\varphi) = 2\varphi - 1.$$



Oscillateur en quadrature. Calculer `sinf()` à chaque échantillon et pour chaque voix coûte cher. Les effets qui n'ont besoin que d'une sinusoïde (chorus, flanger, phaser, ring mod) utilisent un **phaseur tournant** : le point $(c, s) = (\cos\theta, \sin\theta)$ est tourné de l'angle $\Delta = 2\pi f/f_s$ à chaque échantillon par une matrice de rotation,

$$\begin{pmatrix} c \\ s \end{pmatrix} \leftarrow \begin{pmatrix} \cos\Delta & -\sin\Delta \\ \sin\Delta & \cos\Delta \end{pmatrix} \begin{pmatrix} c \\ s \end{pmatrix},$$

soit quatre multiplications et deux additions, et l'on obtient *à la fois* un sinus et un cosinus (deux LFO déphasés de 90° gratuitement). Les erreurs d'arrondi font lentement dériver l'amplitude ; une fois par bloc, on renormalise avec une itération de Newton pour $1/\sqrt{r}$ au voisinage de $r = s^2 + c^2 \approx 1$:

$$k = \tfrac{3}{2} - \tfrac{1}{2}(s^2 + c^2), \qquad (c, s) \leftarrow k\,(c, s).$$

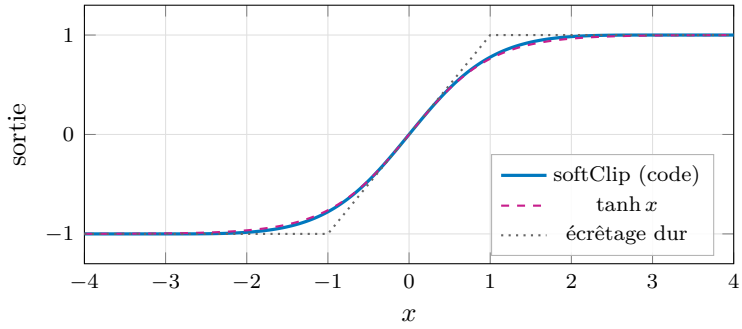
Un déphasage constant ϕ s'obtient sans calcul trigonométrique supplémentaire : $\sin(\theta + \phi) = s\cos\phi + c\sin\phi$ (méthode `sinOffset`).

2.4.5 Saturation douce

Pour empêcher un signal de dépasser la pleine échelle sans « couper » brutalement (ce qui crée des harmoniques aiguës désagréables), on utilise une **saturation douce**. La fonction idéale est tanh, coûteuse ; le code utilise son approximation de Padé

$$\mathrm{softClip}(x) = \begin{cases} x\,\dfrac{27+x^2}{27+9x^2} & |x| \le 3, \\ \mathrm{sgn}(x) & |x| > 3, \end{cases}$$

qui vaut exactement ± 1 en ± 3, se comporte comme x près de 0 et ne demande qu'une division. On la retrouve dans les boucles de réinjection (delay, flanger, phaser) pour qu'elles ne puissent jamais diverger, et dans la distorsion.



2.4.6 Stéréo : Mid/Side et loi de panoramique

Mid/Side. On décompose un signal stéréo en une partie commune (*Mid*) et une partie différentielle (*Side*) :

$$M = \frac{L+R}{2}, \qquad S = \frac{L-R}{2}, \qquad L = M+S, \qquad R = M-S.$$

Multiplier S par un facteur w règle la *largeur stéréo* : $w = 0$ donne du mono, $w = 1$ ne change rien, $w > 1$ élargit l'image (Master).

Panoramique à puissance constante. Pour déplacer un son de gauche à droite sans que son volume perçu baisse au centre, on choisit des gains dont la somme des carrés reste constante (AutoPan) :

$$g_L = \sqrt{\frac{1-p}{2}}, \qquad g_R = \sqrt{\frac{1+p}{2}}, \qquad g_L^2 + g_R^2 = 1, \qquad p \in [-1, 1].$$

CHAPITRE 3

Architecture du traitement

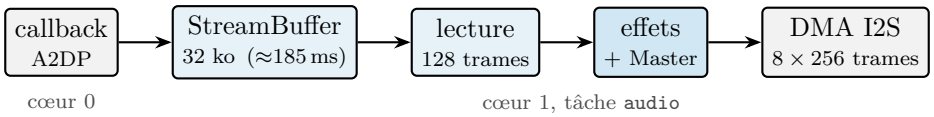
3.1 Deux cœurs, trois rôles

L'ESP32 possède deux cœurs Xtensa à 240 MHz et un système temps réel (FreeRTOS). Le travail est réparti pour que l'audio ne soit *jamais* interrompu par l'affichage :

Cœur	Tâche	Priorité	Rôle
0	pile Bluetooth (Bluedroid)	haute	réception A2DP, décodage SBC, AVRCP (titre, commandes)
0	`BtAppT`	moyenne	événements de la librairie A2DP
1	`audio`	**très haute** (23)	tampon → effets → I2S, en continu
1	`ui`	basse (1)	boutons, potentiomètres, FFT, dessin, écran SPI

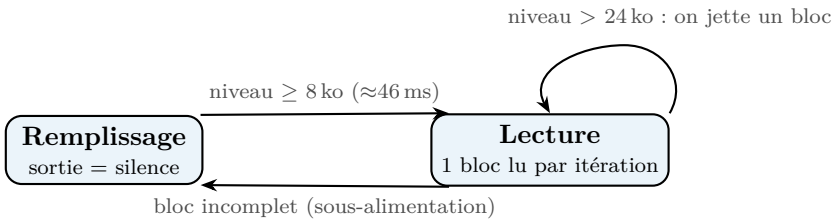
La tâche audio passe l'essentiel de son temps *bloquée* dans `i2s_write()` : elle attend que le DAC ait consommé un bloc. Pendant ce temps, la tâche d'interface (même cœur, priorité plus basse) dessine. Dès qu'un bloc audio peut être écrit, l'ordonnanceur rend la main à l'audio. Mesurée sur la carte, la charge de calcul audio est d'environ 9 % avec les réglages par défaut, et l'interface tourne à environ 33 images par seconde pendant la lecture.

3.2 Chemin des échantillons



3.2.1 Le tampon anti-gigue

Le Bluetooth livre l'audio *par rafales* irrégulières (paquets SBC décodés de quelques millisecondes), alors que le DAC consomme les échantillons à un rythme parfaitement régulier imposé par son horloge. Entre les deux, un **tampon** (`StreamBuffer` FreeRTOS de 32 ko) absorbe les irrégularités. Sa gestion suit une petite machine à états :



— Au démarrage ou après une coupure, on attend 8 ko de données (46 ms) avant de jouer : le tampon a ainsi de la marge pour encaisser les rafales.

— Si le tampon se vide (le téléphone a pris du retard), on repasse en remplissage et on joue du silence. Le compteur *Coupures (vide)* de l'écran Infos s'incrémente.

— Les horloges du téléphone et du DAC ne sont jamais exactement identiques. Si la source est un peu plus rapide, le tampon grossit lentement ; au-delà de 24 ko, on supprime un bloc pour rattraper (compteur *plein*). Cela arrive au plus une fois toutes les quelques minutes.

Même sans musique, la boucle continue de tourner : elle traite des blocs de silence. Ainsi, les queues de réverbération et de delay s'éteignent naturellement quand on met en pause.

3.2.2 Sortie I2S et cadencement

La sortie utilise le contrôleur `I2S_NUM_1` de l'ESP32, en maître, échantillons 32 bits, horloge générée par l'**APLL** (PLL audio dédiée, beaucoup plus précise que la division de l'horloge principale pour 44,1 kHz). Le PCM5102 reconstruit lui-même son horloge interne à partir de BCK (broche SCK reliée à la masse).

Huit tampons DMA de 256 trames (46 ms au total) se vident automatiquement vers le DAC. `i2s_write()` bloque tant qu'aucun tampon n'est libre : c'est ce blocage qui **cadence** toute la boucle audio, sans minuterie.

Attention

Le port `I2S_NUM_0` est volontairement évité : la librairie A2DP reconfigure son horloge à chaque connexion d'un téléphone, même quand sa propre sortie I2S est désactivée.

3.2.3 Latence

La latence ajoutée par le processeur se compose du pré-remplissage (≈46 ms), des tampons DMA (jusqu'à 46 ms) et d'un bloc de traitement (2,9 ms), soit environ 95 ms, à laquelle s'ajoute la latence du codec Bluetooth (souvent 100 ms à 200 ms). Pour écouter de la musique, c'est sans importance ; pour jouer d'un instrument en direct, le Bluetooth serait de toute façon trop lent.

3.2.4 Mesure de la charge

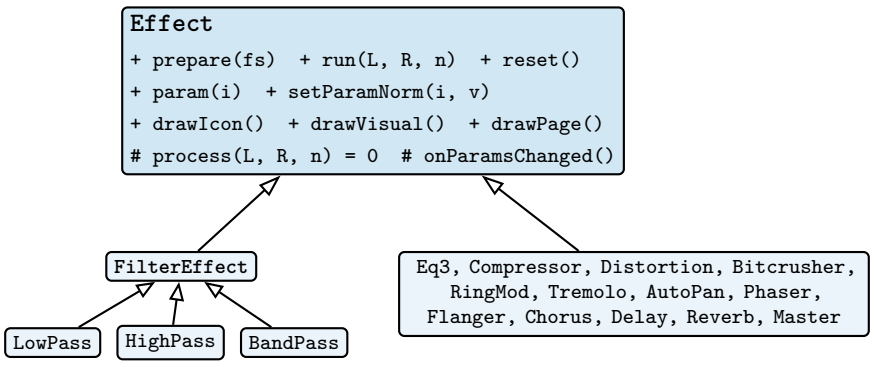
La tâche mesure le temps t_{calc} passé entre la conversion d'entrée et la conversion de sortie, et le compare à la durée d'un bloc T_B :

$$\text{charge} \leftarrow 0{,}95 \cdot \text{charge} + 0{,}05 \cdot \frac{t_{\text{calc}}}{T_B}.$$

Une charge de 100 % signifierait que le calcul prend autant de temps que la durée du son : il faudrait alors désactiver des effets. La valeur s'affiche en haut à droite de l'écran.

3.3 La classe parent `Effect`

Chaque effet est une classe qui hérite de `Effect` (fichiers `src/effects/Effect.h` et `Effect.cpp`). La classe parent fournit tout ce qui est commun : gestion des paramètres, activation sans clic, dessin de la page de réglage. Un effet n'a qu'à décrire son traitement et, s'il le souhaite, sa visualisation.

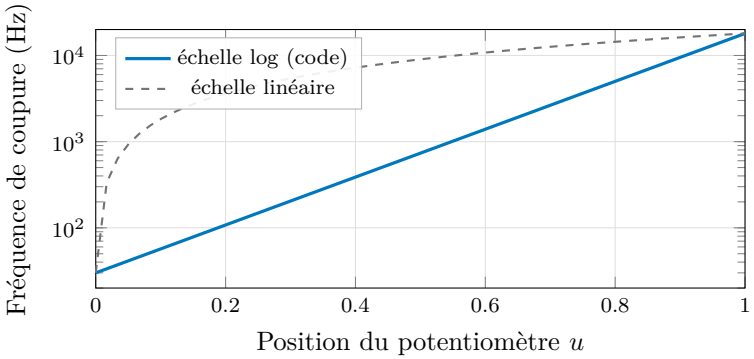


3.3.1 Paramètres

Un effet déclare jusqu'à quatre paramètres dans son constructeur (`addParam`, `addChoice`), un par potentiomètre. Chaque paramètre a un nom, une unité, une plage $[v_{\min}, v_{\max}]$, une valeur par défaut et une **échelle**. Le potentiomètre fournit une position normalisée $u \in [0, 1]$, convertie ainsi :

Échelle	Conversion $u \mapsto v$	Exemples
`Linear`	$v = v_{\min} + u\,(v_{\max} - v_{\min})$	mix, profondeur, gains en dB
`Log`	$v = v_{\min}\,(v_{\max}/v_{\min})^{u}$	fréquences, temps, ratio
`Int`	linéaire, arrondi à l'entier	nombre de bits, facteur de réduction
`Choice`	$v = \mathrm{round}\big(u\,(N-1)\big)$	formes d'onde, pentes, modes

L'échelle logarithmique est essentielle pour les fréquences : l'oreille perçoit les *rapports* de fréquences (une octave = $\times 2$). Avec une échelle linéaire de 30 Hz à 18 000 Hz, toute la moitié basse du potentiomètre couvrirait seulement les fréquences au-dessus de 9 kHz !

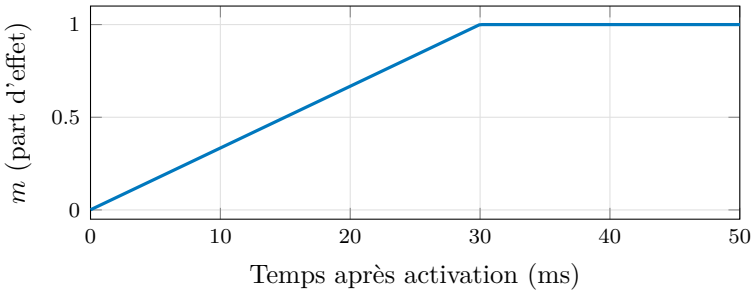


Partage entre tâches. Les paramètres sont écrits par la tâche d'interface et lus par la tâche audio. Pour éviter tout verrou, chaque modification incrémente un *compteur de version*. Au début de chaque bloc, `run()` compare ce compteur à la dernière version vue et, s'il a changé, appelle `onParamsChanged()` *dans la tâche audio*. Les coefficients des filtres ne sont donc jamais modifiés pendant qu'un échantillon est calculé.

3.3.2 Activation sans clic

Couper ou rétablir brutalement un effet crée une discontinuité, entendue comme un « clic ». `run()` réalise donc un **fondu enchaîné** de 30 ms entre le signal sec (non traité) et le signal traité, piloté par une variable $m \in [0, 1]$:

$$y[n] = x[n] + m[n]\,\big(y_{\text{effet}}[n] - x[n]\big), \qquad m[n] = \mathrm{clamp}\Big(m[n-1] \pm \frac{1}{0{,}03\,f_s},\ 0,\ 1\Big).$$



Trois cas évitent tout calcul inutile :

— effet désactivé et fondu terminé ($m = 0$) : `run()` retourne immédiatement, coût nul ;

— effet actif et $m = 1$: seul `process()` est appelé ;

— effet qui vient d'être activé ($m = 0$) : `reset()` vide d'abord ses mémoires, pour ne pas rejouer un vieil écho resté dans un tampon.

Dans le code — `Effect::run()`

```
void Effect::run(float* L, float* R, int n) {
  const uint32_t v = version_;
  if (v != seenVersion_) { seenVersion_ = v; onParamsChanged(); }

  const bool on = enabled_;
  if (!on && mix_ <= 0.f) return;                        // bypass complet
  if (on && mix_ >= 1.f) { process(L, R, n); return; }  // effet seul
  if (on && mix_ <= 0.f) reset();                        // reprise : buffers propres

  static float dryL[cfg::BLOCK], dryR[cfg::BLOCK];       // copie du signal sec
  memcpy(dryL, L, n * sizeof(float));
  memcpy(dryR, R, n * sizeof(float));
  process(L, R, n);
  const float step = (on ? 1.f : -1.f) / (0.03f * fs_); // fondu de 30 ms
  float m = mix_;
  for (int i = 0; i < n; ++i) {
    m = dsp::clampf(m + step, 0.f, 1.f);
    L[i] = dryL[i] + (L[i] - dryL[i]) * m;
    R[i] = dryR[i] + (R[i] - dryR[i]) * m;
  }
  mix_ = m;
}
```

3.3.3 Affichage

La classe parent dessine la page complète d'un effet (`drawPage`) : un en-tête dans la couleur de l'effet avec son icône animée et son interrupteur, une zone de visualisation, puis quatre boutons rotatifs. Les classes enfants personnalisent :

— `drawIcon()` : la petite icône animée (liste des effets, chaîne, en-tête) ;

— `drawVisual()` : la zone centrale. La classe parent fournit des aides : `drawResponse()` (courbe de réponse en fréquence sur échelle log), `drawLfo()` (forme d'onde avec curseur de phase), `drawTransfer()` (courbe entrée/sortie et sinus déformé). Le spectre de sortie est dessiné en transparence derrière.

Prise au vol des potentiomètres. En arrivant sur la page d'un effet, la position physique des potentiomètres ne correspond pas aux valeurs des paramètres. Pour éviter un saut, un potentiomètre ne pilote son paramètre qu'après avoir été tourné (variation supérieure à 1,2 %). En attendant, un point orange sur le bouton rotatif indique sa position physique.

3.3.4 Ajouter un effet

Dans le code — squelette d'un nouvel effet

```
class MonEffet : public Effect {
 public:
  enum { P_GAIN, P_MIX };
  MonEffet() : Effect("Mon effet", gfx::rgb(255, 0, 128)) {
    addParam("Gain", "dB", -12.f, 12.f, 0.f);   // potentiometre 1
    addParam("Mix", "%", 0.f, 1.f, 0.5f);       // potentiometre 2
  }
 protected:
```

```
  void onParamsChanged() override { g_ = dsp::dbToGain(p(P_GAIN)); }
  void process(float* L, float* R, int n) override {
    const float mix = p(P_MIX);
    for (int i = 0; i < n; ++i) {
      L[i] += (L[i] * g_ - L[i]) * mix;
      R[i] += (R[i] * g_ - R[i]) * mix;
    }
  }
  float g_ = 1.f;
};
```

On place la classe dans `src/effects/MonEffet.h`, on l'inclut dans `AllEffects.h` et on l'ajoute à la liste de `AudioEngine::begin()`. Elle apparaît alors dans le menu, avec sa page, ses quatre boutons rotatifs et son fondu d'activation.

3.4 Ordre de la chaîne

L'ordre initial suit les usages des studios :

HPF LPF BPF EQ Comp Dist Crush Ring Trem Pan Phaser Flanger Chorus Delay Reverb

On met en forme le timbre d'abord (filtres, égaliseur), on contrôle la dynamique avant de saturer (le compresseur égalise le niveau qui attaque la distorsion), on module ensuite, et on termine par les effets d'espace (delay puis reverb) : une réverbération placée avant une distorsion produirait une bouillie saturée. L'écran *Chaîne* permet de réordonner les effets actifs ; l'échange se fait sous verrou très court, et la tâche audio copie la liste des effets au début de chaque bloc.

CHAPITRE 4

Filtres et égalisation

Les filtres modifient le **timbre** : ils renforcent ou atténuent certaines fréquences sans créer de fréquences nouvelles. Tous les effets de ce chapitre reposent sur le biquad (section 2.4.2).

4.1 La classe intermédiaire `FilterEffect`

Fichier : `src/effects/FilterEffect.h`

Les trois filtres résonants (passe-bas, passe-haut, passe-bande) ne diffèrent que par la formule des coefficients. Ils partagent donc une classe intermédiaire `FilterEffect`, dont les classes `LowPassEffect`, `HighPassEffect` et `BandPassEffect` ne font que choisir le type, le nom, la couleur et la fréquence par défaut :

Dans le code — `LowPassEffect.h` (fichier complet)

```
class LowPassEffect : public FilterEffect {
 public:
  LowPassEffect()
      : FilterEffect("Low Pass", gfx::rgb(0, 170, 255), dsp::BiquadType::LowPass, 1200.f) {}
};
```

4.1.1 Paramètres communs

Pot	Nom	Plage	Échelle	Rôle
1	**Cutoff**	30 Hz à 18 000 Hz	log	fréquence de coupure f_0
2	**Reso**	Q de 0,5 à 10 (défaut 0,9)	log	résonance : bosse autour de f_0
3	**Pente**	12 dB ou 24 dB/oct.	choix	un ou deux biquads en cascade
4	**LFO**	0 Hz à 8 Hz (0 = arrêt)	linéaire	balayage automatique de f_0

4.1.2 Modèle : du filtre analogique au biquad

Les filtres résonants dérivent du prototype analogique du second ordre, par exemple pour le passe-bas :

$$H_a(s) = \frac{1}{(s/\Omega_0)^2 + (s/\Omega_0)/Q + 1}.$$

La **transformation bilinéaire**, $s \to \frac{2}{T}\frac{1-z^{-1}}{1+z^{-1}}$, avec une pré-distorsion de fréquence qui garantit que la coupure tombe exactement à f_0, donne les formules du *Audio EQ Cookbook*. Avec $\omega_0 = 2\pi f_0/f_s$ et $\alpha = \sin\omega_0/(2Q)$:

Type	b_0	b_1	b_2	a_0	a_1	a_2
Passe-bas	$\frac{1-\cos\omega_0}{2}$	$1-\cos\omega_0$	$\frac{1-\cos\omega_0}{2}$	$1+\alpha$	$-2\cos\omega_0$	$1-\alpha$
Passe-haut	$\frac{1+\cos\omega_0}{2}$	$-(1+\cos\omega_0)$	$\frac{1+\cos\omega_0}{2}$	$1+\alpha$	$-2\cos\omega_0$	$1-\alpha$
Passe-bande	α	0	$-\alpha$	$1+\alpha$	$-2\cos\omega_0$	$1-\alpha$

Tous les coefficients sont ensuite divisés par a_0, pour que le dénominateur commence par 1.

Ce que fait la résonance. Pour le passe-bas et le passe-haut, le gain à la fréquence de coupure vaut exactement Q :

$$|H(e^{j\omega_0})| = Q \qquad \Longrightarrow \qquad Q = 4 \text{ donne une bosse de } 20\log_{10} 4 \approx 12\,\text{dB}.$$

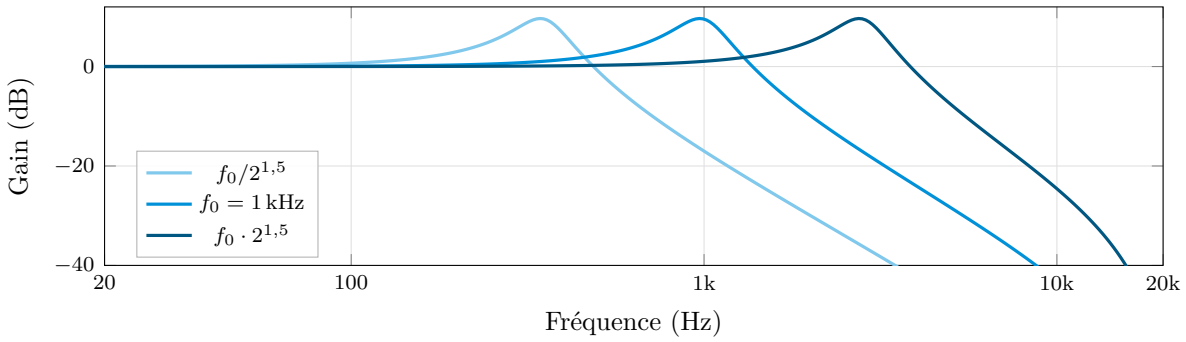
Avec $Q = 1/\sqrt{2} \approx 0{,}707$ (réponse de Butterworth), la courbe est la plus plate possible, sans bosse, et vaut $-3\,$dB à la coupure.

Pente 24 dB/octave. Au-delà de la coupure, un biquad atténue de 12 dB par octave (second ordre). En mode **24dB**, un second biquad identique mais à $Q = 0{,}707$ est placé en cascade. Les réponses (en dB) s'additionnent : la pente double, et la résonance reste réglée par le premier étage seulement, ce qui évite une bosse au carré, assourdissante.

Filtre automatique (LFO). Si **LFO** $> 0{,}05\,$Hz, la coupure oscille d'une octave et demie de part et d'autre de la valeur réglée :

$$f_0(t) = f_{\text{Cutoff}} \cdot 2^{\,1{,}5\,\sin(2\pi f_{\text{LFO}} t)}.$$

Sur un passe-bande à forte résonance, c'est l'effet *auto-wah* ; sur un passe-bas, un « balayage » typique de la musique électronique.



Excursion du LFO sur un passe-bas résonant ($Q = 3$) : la bosse balaie environ trois octaves.

4.1.3 Implémentation

Dans le code — `FilterEffect::process()`

```
const float rate = p(P_LFO);
float target = p(P_FREQ);
if (rate > 0.05f) {
  lfo_.setRate(rate, fs_);
  lfo_.advance(n);                                  // une mise a jour par bloc
  target *= exp2f(1.5f * lfo_.sine());
}
// lissage logarithmique de la coupure (evite les "zips")
const float nf = expf(logf(curFreq_) + (logf(target) - logf(curFreq_)) * 0.35f);
if (fabsf(nf - curFreq_) > 0.01f) { curFreq_ = nf; updateCoefs(); }

const bool steep = choice(P_SLOPE) == 1;
for (int i = 0; i < n; ++i) {
  float l = st_[0][0].process(L[i], c1_), r = st_[0][1].process(R[i], c1_);
  if (steep) { l = st_[1][0].process(l, c2_); r = st_[1][1].process(r, c2_); }
  L[i] = l; R[i] = r;
}
```

— **Lissage de la coupure.** Tourner un potentiomètre fait varier la coupure par paliers (le potentiomètre est lu 33 fois par seconde). Recalculer brutalement les coefficients produirait

un crépitement. La coupure se rapproche donc de sa cible de 35 % par bloc, *en échelle logarithmique*, soit une constante de temps d'environ $-T_B/\ln(0{,}65) \approx 7\,\text{ms}$.

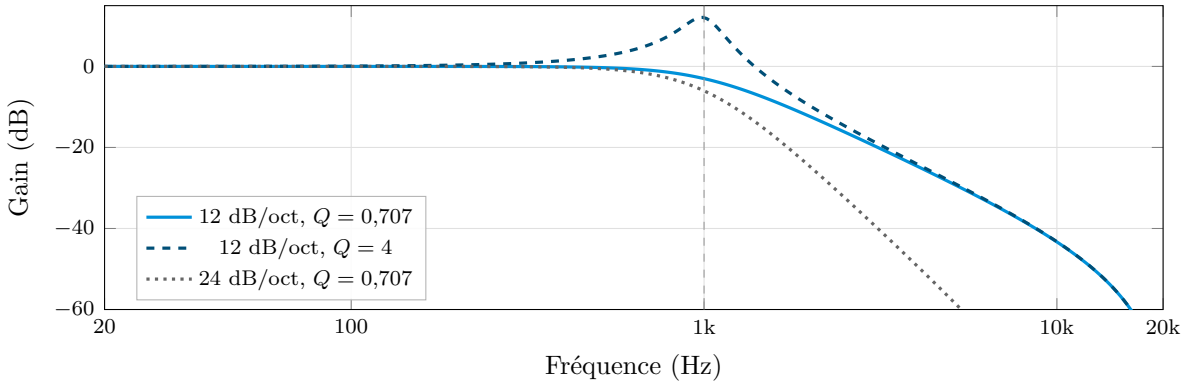
— **Coût.** Un biquad coûte 5 multiplications et 4 additions par échantillon et par canal. Les coefficients (un `sinf`, un `cosf`, une division) ne sont recalculés qu'une fois par bloc, et seulement si la coupure bouge.

— **Visualisation.** La page affiche la réponse en fréquence calculée à partir des mêmes coefficients, avec un marqueur à la fréquence de coupure courante (qui se déplace avec le LFO).

4.2 Low Pass (passe-bas)

Fichier : `src/effects/LowPassEffect.h`

Le passe-bas garde les graves et retire les aigus : le son devient *sourd*, *étouffé*, comme entendu à travers un mur. Avec de la résonance, une note sifflante apparaît à la coupure : c'est le son caractéristique des synthétiseurs analogiques.



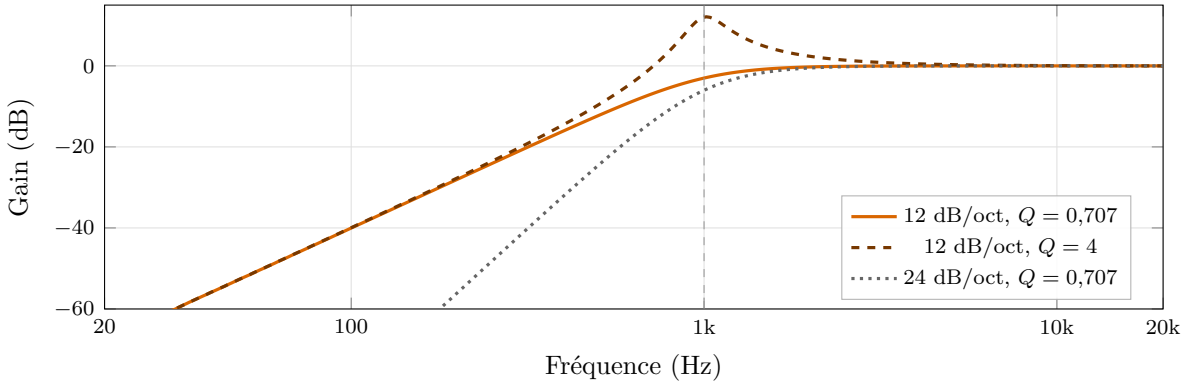
Réponses du passe-bas à $f_0 = 1\,\text{kHz}$, calculées avec `BiquadCoefs::design`. Au-delà de la coupure, la pente est de $-12\,\text{dB}$ puis $-24\,\text{dB}$ par octave.

Usages. Adoucir un enregistrement trop brillant ; simuler un son lointain, un téléphone ou une pièce voisine ; « ouvrir » progressivement un morceau en tournant la coupure du grave vers l'aigu. Réglage par défaut : 1 200 Hz.

4.3 High Pass (passe-haut)

Fichier : `src/effects/HighPassEffect.h`

Le passe-haut est le symétrique : il retire les graves. Le son devient fin, léger, « petit transistor ». C'est l'effet idéal pour faire disparaître la basse avant un retour fracassant.



Le passe-haut est placé en **tête de chaîne** : couper les infra-graves inutiles (30–60 Hz) avant la distorsion ou la réverbération évite qu'elles « boursouflent » le son. Réglage par défaut : 400 Hz.

4.4 Band Pass (passe-bande)

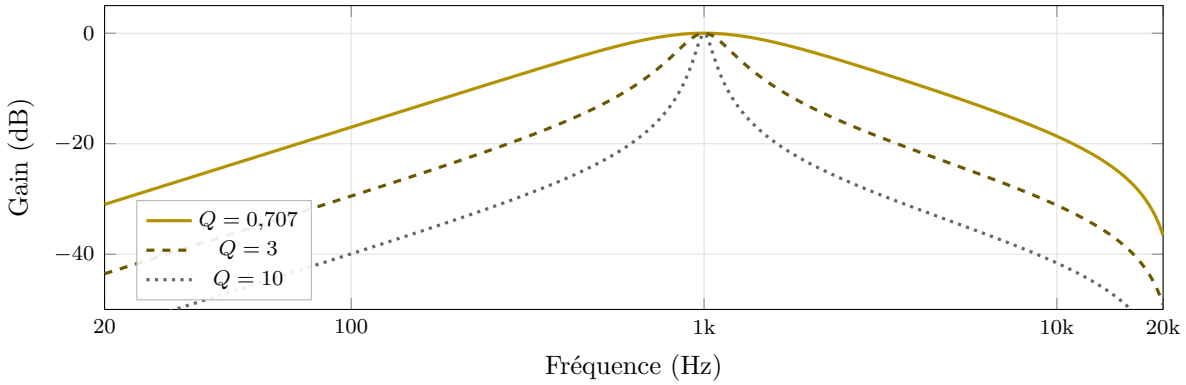
Fichier : `src/effects/BandPassEffect.h`

> Le passe-bande ne garde qu'une bande de fréquences autour de f_0 : effet « radio », « mégaphone », ou voix au téléphone. Avec une forte résonance et le LFO, on obtient un *wah-wah* automatique.

La variante utilisée a un **gain de** 0 dB **au sommet**, quel que soit Q. La largeur de la bande à -3 dB vaut environ

$$\Delta f \approx \frac{f_0}{Q}.$$

Augmenter la résonance *rétrécit* donc la bande sans changer le niveau au centre : le son paraît plus faible car il reste moins d'énergie.



En mode **24dB**, le second étage ($Q = 0{,}707$) rend les flancs deux fois plus raides. Réglage par défaut : 900 Hz.

4.5 EQ 3 bandes

Fichier : `src/effects/Eq3Effect.h`

> L'égaliseur est la tonalité « grave / médium / aigu » d'une chaîne hi-fi, en plus précis. Chaque bande peut être renforcée ou atténuée de 15 dB, et la fréquence centrale du médium est réglable.

4.5.1 Paramètres

Pot	Nom	Plage	Échelle	Filtre
1	**Grave**	-15 dB à 15 dB	lin.	plateau bas (*low shelf*) à 150 Hz, $Q = 0{,}7$
2	**Medium**	-15 dB à 15 dB	lin.	cloche (*peak*), $Q = 0{,}9$
3	**Freq**	200 Hz à 5 000 Hz	log	centre de la cloche (défaut 1 kHz)
4	**Aigu**	-15 dB à 15 dB	lin.	plateau haut (*high shelf*) à 5 kHz, $Q = 0{,}7$

4.5.2 Modèle

Les trois biquads sont placés en série. Leurs gains en dB s'additionnent :

$$G_{\text{total}}(f) = G_{\text{grave}}(f) + G_{\text{médium}}(f) + G_{\text{aigu}}(f).$$

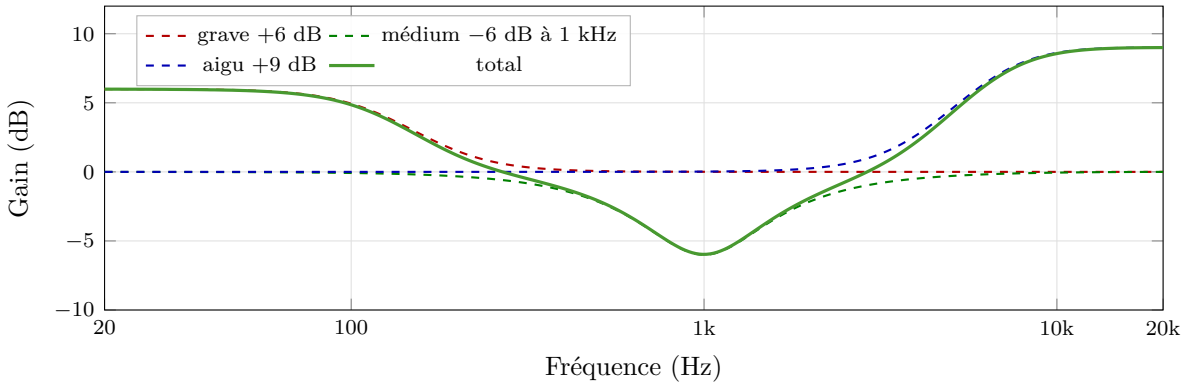
Avec $A = 10^{G/40}$ (racine du gain linéaire) et $\alpha = \sin\omega_0/(2Q)$, la cloche s'écrit

$$H_{\text{peak}}(z) = \frac{(1+\alpha A) - 2\cos\omega_0\, z^{-1} + (1-\alpha A)z^{-2}}{(1+\alpha/A) - 2\cos\omega_0\, z^{-1} + (1-\alpha/A)z^{-2}}.$$

À f_0, son gain vaut exactement $A^2 = 10^{G/20}$, c'est-à-dire G dB ; loin de f_0, il vaut 1. Les plateaux (*shelving*) ont un gain G d'un côté de f_0 et 0 dB de l'autre :

$$\begin{aligned} H_{\text{low}}: \quad b_0 &= A\big[(A+1) - (A-1)\cos\omega_0 + 2\sqrt{A}\,\alpha\big], & a_0 &= (A+1) + (A-1)\cos\omega_0 + 2\sqrt{A}\,\alpha, \\ b_1 &= 2A\big[(A-1) - (A+1)\cos\omega_0\big], & a_1 &= -2\big[(A-1) + (A+1)\cos\omega_0\big], \\ b_2 &= A\big[(A+1) - (A-1)\cos\omega_0 - 2\sqrt{A}\,\alpha\big], & a_2 &= (A+1) + (A-1)\cos\omega_0 - 2\sqrt{A}\,\alpha, \end{aligned}$$

et le plateau haut s'obtient en changeant le signe des termes en $\cos\omega_0$ (voir le code).



Exemple de réglage « en V » (grave et aigu renforcés, médium creusé) et contribution de chaque bande.

4.5.3 Implémentation et visualisation

Les coefficients sont recalculés uniquement dans `onParamsChanged()`, donc seulement quand un potentiomètre bouge. Le traitement coûte trois biquads par canal (15 multiplications par échantillon). La page affiche la courbe totale et trois poignées colorées (rouge, vert, bleu), placées à la fréquence et au gain de chaque bande.

Attention

Renforcer fortement le grave ou l'aigu augmente le niveau crête : avec 15 dB, le signal dépasserait la pleine échelle. Le limiteur du Master (section 8.1) protège la sortie, mais il vaut mieux *creuser* ce qui gêne que tout renforcer.

CHAPITRE 5

Dynamique et distorsion

Les effets de ce chapitre sont **non linéaires** : leur sortie n'est pas proportionnelle à leur entrée. Ils agissent sur le niveau (compresseur) ou créent des fréquences qui n'existaient pas (distorsion, bitcrusher, modulateur en anneau).

5.1 Compressor

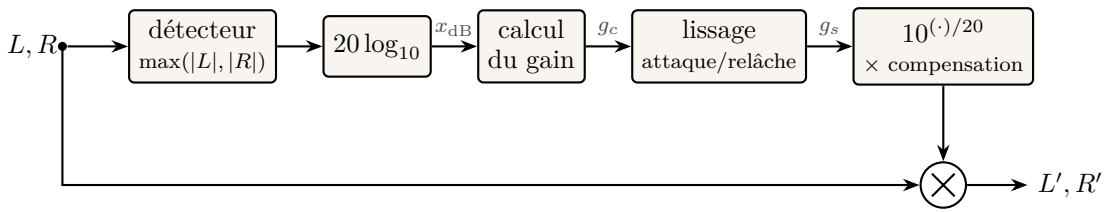
Fichier : `src/effects/CompressorEffect.h`

> Un compresseur est un « réglage de volume automatique » : quand le son devient fort, il baisse le gain ; quand le son redevient faible, il le remonte. L'écart entre passages forts et faibles (la *dynamique*) se réduit, le mix paraît plus dense et plus présent.

5.1.1 Paramètres

Pot	Nom	Plage	Échelle	Rôle
1	**Seuil**	−40 dB à 0 dB (déf. −18)	lin.	niveau à partir duquel on comprime
2	**Ratio**	1 :1 à 20 :1 (déf. 4 :1)	log	force de la compression
3	**Attaque**	0,1 ms à 100 ms (déf. 5)	log	vitesse de réaction quand le son monte
4	**Relache**	10 ms à 1 000 ms (déf. 120)	log	vitesse de retour quand le son baisse

5.1.2 Schéma-bloc



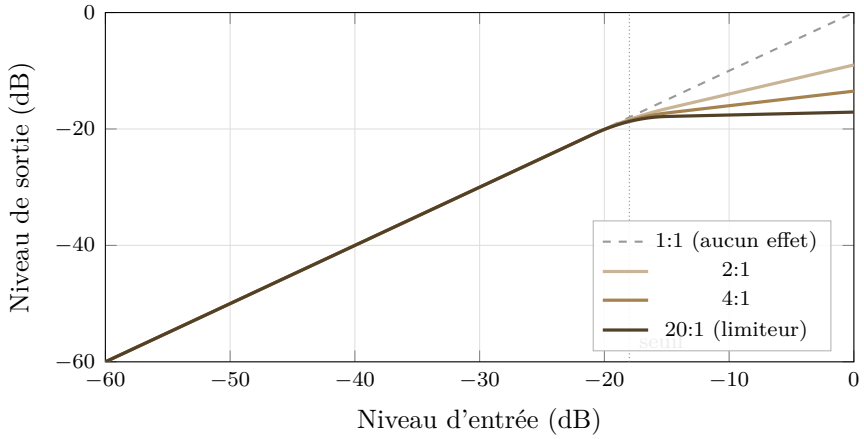
Le compresseur est **stéréo lié** : un seul gain, calculé sur le plus fort des deux canaux, est appliqué aux deux. Comprimer chaque canal séparément ferait bouger l'image stéréo (un instrument fort à gauche ferait « glisser » le mix vers la droite).

5.1.3 Courbe statique et genou doux

Le **calculateur de gain** travaille en décibels. Avec le seuil T, le ratio R et une largeur de genou $W = 6\,\text{dB}$, le niveau de sortie visé pour un niveau d'entrée x (en dB) est

$$y(x) = \begin{cases} x & \text{si } x - T \le -W/2 \quad \text{(sous le seuil : rien ne change)}, \\ T + \dfrac{x - T}{R} & \text{si } x - T \ge W/2 \quad \text{(au-dessus : pente } 1/R\text{)}, \\ x + \left(\dfrac{1}{R} - 1\right)\dfrac{(x - T + W/2)^2}{2W} & \text{sinon (genou : raccord parabolique).} \end{cases}$$

Le gain à appliquer est $g_c = y(x) - x \leq 0$. Le *genou doux* raccorde les deux droites par une parabole : la compression s'installe progressivement autour du seuil, ce qui est beaucoup moins audible qu'une cassure nette.



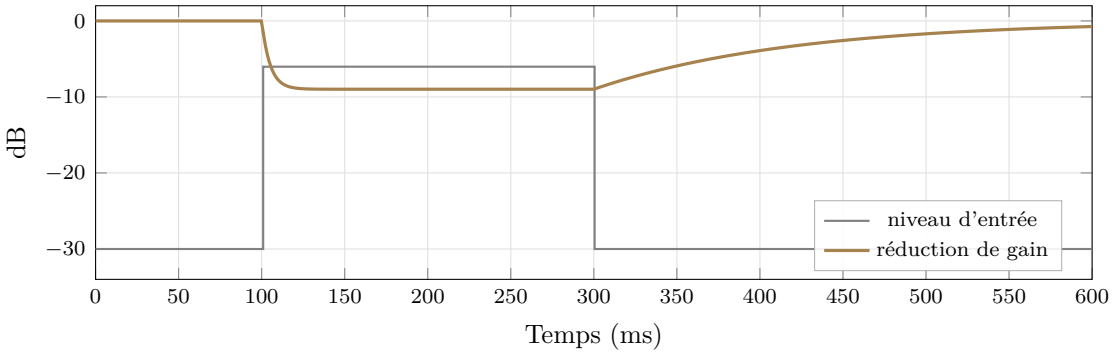
Courbes statiques réelles (`gainComputer()`), seuil −18 dB, genou de 6 dB.

5.1.4 Attaque et relâche

Appliquer g_c directement ferait varier le gain à chaque oscillation du signal : on obtiendrait une distorsion. Le gain est donc **lissé** par un filtre à un pôle (section 2.4.1) dont la constante de temps dépend du sens de variation :

$$g_s[n] = g_c[n] + \alpha\,(g_s[n-1] - g_c[n]), \qquad \alpha = \begin{cases} \alpha_A = \mathrm{e}^{-1/(\tau_A f_s)} & \text{si } g_c[n] < g_s[n-1] \text{ (plus de réduction)} \\ \alpha_R = \mathrm{e}^{-1/(\tau_R f_s)} & \text{sinon.} \end{cases}$$

Le lissage se fait **dans le domaine des décibels** : la réduction de gain évolue alors à vitesse constante en dB, ce que l'oreille perçoit comme un mouvement régulier.



Mesure sur le vrai code : le niveau passe de −30 dB à −6 dB pendant 200 ms (seuil −18 dB, 4 :1, attaque 5 ms, relâche 120 ms). La réduction atteint $-(12 - 12/4) = -9$ dB en quelques millisecondes, puis revient lentement.

Compensation automatique. Comprimer baisse le niveau moyen. Pour comparer à l'oreille avec et sans compresseur, un gain fixe est ajouté :

$$M = -\frac{T\,(1 - 1/R)}{2} \text{ dB} \qquad (\text{exemple : } T = -18,\ R = 4 \Rightarrow M = +6{,}75\,\text{dB}).$$

C'est la moitié de la réduction subie par un signal à 0 dBFS, un compromis courant.

5.1.5 Implémentation

Dans le code — `CompressorEffect::process()`

```
for (int i = 0; i < n; ++i) {
  const float a = fmaxf(fabsf(L[i]), fabsf(R[i]));          // detection crete, stereo liee
  const float inDb = dsp::gainToDb(a);
  const float gr = gainComputer(inDb) - inDb;               // <= 0
  env = gr < env ? gr + (env - gr) * att_ : gr + (env - gr) * rel_;
  const float g = dsp::dbToGain(env) * makeup_;
  L[i] *= g;  R[i] *= g;
}
```

— **Coût.** Un logarithme et une exponentielle par échantillon : c'est l'un des effets les plus coûteux par échantillon, mais cela reste de l'ordre de 1 % du processeur.

— **Détection crête.** Le détecteur suit la valeur absolue instantanée. Combiné à une attaque très courte, il peut suivre la forme d'onde des basses : choisir une attaque d'au moins quelques millisecondes et une relâche supérieure à la période des graves (>50 ms).

— **Visualisation.** La page trace la courbe statique, un point qui se déplace au niveau d'entrée courant, et un vumètre de réduction de gain.

5.2 Distortion

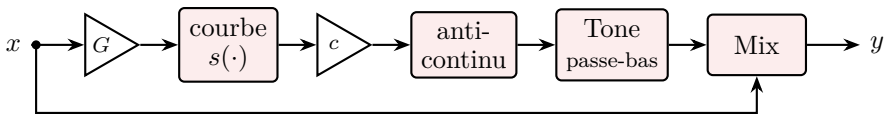
Fichier : `src/effects/DistortionEffect.h`

Une distorsion amplifie fortement le signal puis l'écrase pour qu'il ne dépasse pas une limite, comme un ampli de guitare poussé à fond. La forme d'onde s'arrondit ou devient carrée, ce qui ajoute des *harmoniques* : le son devient plus riche, plus agressif.

5.2.1 Paramètres

Pot	Nom	Plage	Échelle	Rôle
1	**Drive**	0 dB à 40 dB (déf. 18)	lin.	gain avant saturation $G = 10^{\text{Drive}/20}$
2	**Tone**	800 Hz à 12 000 Hz (déf. 5000)	log	passe-bas après saturation
3	**Type**	Douce, Dure, Fuzz, Fold	choix	courbe de saturation
4	**Mix**	0 % à 100 % (déf. 100)	lin.	dosage avec le son original

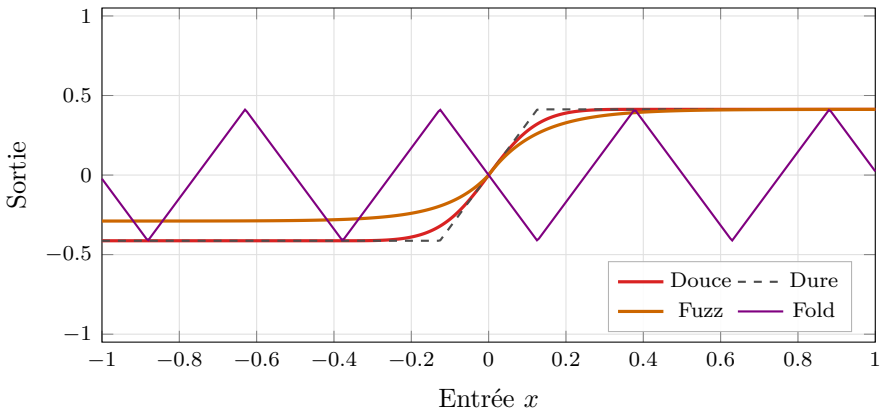
5.2.2 Chaîne de traitement



5.2.3 Les quatre courbes

Avec $u = G\,x$:

$$\begin{aligned}
\text{Douce :}\quad & s(u) = \text{softClip}(u) \quad (\textit{section}\ 2.4.5)\\
\text{Dure :}\quad & s(u) = \text{clamp}(u,\,-1,\,1)\\
\text{Fuzz :}\quad & s(u) = \begin{cases} 1 - \mathrm{e}^{-u} & u > 0 \\ -0{,}7\,(1 - \mathrm{e}^{1{,}12\,u}) & u \le 0 \end{cases}\\
\text{Fold :}\quad & s(u) = 4\left|\left\{\tfrac{u+1}{4}\right\} - \tfrac{1}{2}\right| - 1 \qquad (\{\cdot\} : \text{partie fractionnaire})
\end{aligned}$$



Courbes de transfert réelles avec Drive = 18 dB, compensation de niveau incluse.

— **Douce** : saturation progressive, proche d'un ampli à lampes.

— **Dure** : écrêtage net. Au-delà de la limite, la forme d'onde est coupée à plat.

— **Fuzz** : *asymétrique*. Les alternances positives saturent à 1, les négatives à $-0{,}7$, comme les pédales à transistors des années 1960.

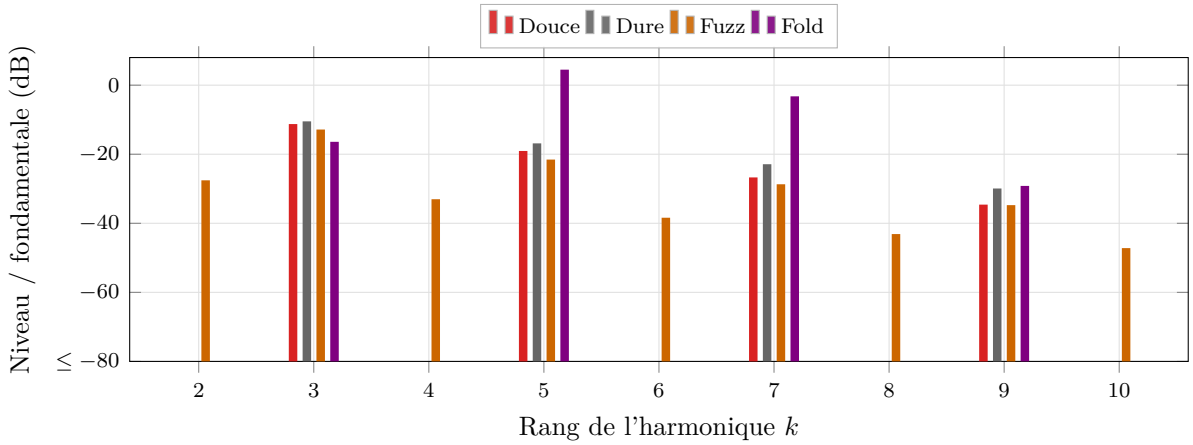
— **Fold** (*wavefolder*) : au lieu de plafonner, le signal est *replié* vers l'intérieur. Plus le drive est élevé, plus la forme d'onde se replie, ce qui donne un son métallique typique des synthétiseurs « West Coast ».

5.2.4 Pourquoi des harmoniques ?

Si l'entrée est une sinusoïde de fréquence f_0, la sortie $s(G \sin 2\pi f_0 t)$ est encore périodique de période $1/f_0$, mais n'est plus sinusoïdale. Sa série de Fourier ne contient que des multiples de f_0 : les **harmoniques** $2f_0, 3f_0, \ldots$

— Une courbe **impaire** ($s(-u) = -s(u)$: douce, dure, fold) traite de la même façon les alternances positives et négatives. Elle ne produit que des **harmoniques impaires** ($3f_0, 5f_0, \ldots$), au son « creux » et chaud.

— Une courbe **asymétrique** (fuzz) produit aussi des **harmoniques paires** ($2f_0, 4f_0, \ldots$), à l'octave de la note, ainsi qu'une composante continue.



Harmoniques mesurées par FFT sur la sortie du vrai effet (sinus 1 kHz, amplitude 0,5, Drive 18 dB). Seule la fuzz produit les rangs pairs. Le fold renvoie plus d'énergie dans l'harmonique 5 que dans la fondamentale.

5.2.5 Étages auxiliaires

Compensation de niveau. Sans correction, monter le drive rendrait l'effet de plus en plus fort. La sortie est multipliée par

$$c(G) = 0{,}2 + \frac{0{,}6}{\sqrt{G}},$$

une loi empirique qui garde un volume perçu à peu près constant.

Bloqueur de continu. La fuzz décale la moyenne du signal. Une composante continue gaspille de la marge, fait « claquer » le DAC à l'activation et perturbe les effets suivants. On l'élimine avec le passe-haut classique

$$y[n] = x[n] - x[n-1] + 0{,}995\, y[n-1], \qquad H(z) = \frac{1 - z^{-1}}{1 - 0{,}995\, z^{-1}},$$

dont la coupure vaut $f_c \approx \frac{(1-0{,}995)\, f_s}{2\pi} \approx 35$ Hz.

Tone. Un passe-bas à un pôle adoucit les harmoniques les plus aiguës, qui sont les plus agressives.

Attention

Repliement. Les harmoniques créées au-delà de 22 kHz se replient dans la bande audible (chapitre 2), à des fréquences non harmoniques : le son prend un grain « numérique » aux réglages extrêmes. Le remède habituel est le *suréchantillonnage*, qui consiste à calculer la saturation à $4f_s$ puis à filtrer. Il n'est pas implémenté ici, pour économiser le calcul. Baisser le **Tone** masque une partie du phénomène.

5.3 Bitcrusher

Fichier : `src/effects/BitcrusherEffect.h`

Le bitcrusher fait sonner la musique comme une vieille console de jeux ou un échantillonneur des années 1980, en dégradant volontairement les deux qualités d'un signal numérique : la **résolution** (nombre de bits) et la **fréquence d'échantillonnage**.

5.3.1 Paramètres

Pot	Nom	Plage	Échelle	Rôle
1	**Bits**	1 à 16 (déf. 6)	entier	résolution simulée
2	**Reduc**	×1 à ×32 (déf. ×4)	entier	division de la fréquence d'échantillonnage
3	**Mix**	0 % à 100 % (déf. 100)	lin.	dosage

5.3.2 Quantification

Avec b bits, on ne garde que $Q = 2^{b-1}$ niveaux par polarité, espacés de $\Delta = 1/Q$:

$$y = \frac{\operatorname{round}(Q\,x)}{Q}.$$

L'erreur commise, de $\pm\Delta/2$, se comporte comme un bruit de puissance $\Delta^2/12$. Pour une sinusoïde pleine échelle, le rapport signal sur bruit vaut

$$\mathrm{RSB} \approx 6{,}02\,b + 1{,}76 \text{ dB},$$

soit 98 dB à 16 bits (qualité CD), 38 dB à 6 bits (souffle très présent) et 8 dB à 1 bit (signal carré). Aux très faibles résolutions, l'erreur n'est plus un bruit mais une distorsion très audible, corrélée au signal.

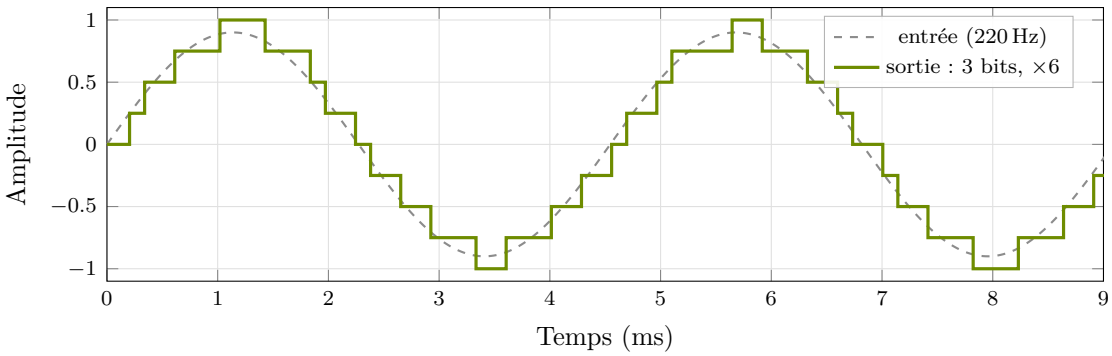
5.3.3 Réduction d'échantillonnage

On ne prélève qu'un échantillon sur D et on le maintient pendant D échantillons (*échantillonneur-bloqueur*) :

$$y[n] = x[D\lfloor n/D \rfloor].$$

Tout se passe comme si la fréquence d'échantillonnage valait f_s/D (11 kHz pour $D = 4$). Comme on n'applique aucun filtre anti-repliement, les aigus au-delà de $f_s/(2D)$ se replient en fréquences parasites, et les marches du maintien créent des *images* du spectre. C'est précisément le grain recherché. Le maintien a la réponse en fréquence d'un sinus cardinal :

$$|H(f)| = \left| \frac{\sin(\pi f D/f_s)}{D\,\sin(\pi f/f_s)} \right|.$$



Sortie mesurée du vrai effet. Les niveaux sont espacés de $\Delta = 1/4$ et chaque palier dure 6 échantillons.

Dans le code — `BitcrusherEffect::process()`

```
const float levels = exp2f(p(P_BITS) - 1.f), inv = 1.f / levels;
for (int i = 0; i < n; ++i) {
  if (++cnt_ >= ds) {                    // nouvel echantillon tous les ds
    cnt_ = 0;
```

```
    hold_[0] = roundf(L[i] * levels) * inv; // quantification
    hold_[1] = roundf(R[i] * levels) * inv;
  }
  L[i] += (hold_[0] - L[i]) * mix;
  R[i] += (hold_[1] - R[i]) * mix;
}
```

C'est l'un des effets les moins coûteux : un arrondi et deux multiplications par échantillon.

5.4 Ring Mod (modulateur en anneau)

Fichier : `src/effects/RingModEffect.h`

> Le modulateur en anneau multiplie la musique par une onde, la *porteuse*. Les notes disparaissent au profit de fréquences inharmoniques : son de cloche, de métal, voix de robot (les Daleks de *Doctor Who* !).

5.4.1 Paramètres

Pot	Nom	Plage	Échelle	Rôle
1	**Freq**	20 Hz à 3 000 Hz (déf. 440)	log	fréquence de la porteuse f_c
2	**Forme**	Sinus, Triangle, Carré	choix	forme de la porteuse
3	**Mix**	0 % à 100 % (déf. 70)	lin.	dosage

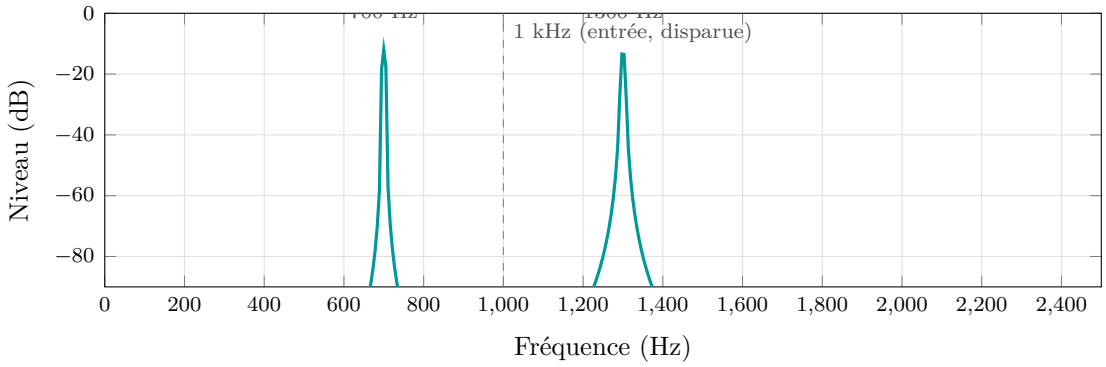
5.4.2 Modèle

$$y(t) = x(t) \cdot c(t), \qquad \text{dosé} : y = x + \text{Mix}\,(x\,c - x).$$

Pour une entrée sinusoïdale de fréquence f_x et une porteuse sinusoïdale de fréquence f_c, l'identité trigonométrique

$$\sin(2\pi f_x t)\,\sin(2\pi f_c t) = \tfrac{1}{2}\left[\cos 2\pi(f_x - f_c)t - \cos 2\pi(f_x + f_c)t\right]$$

montre que la fréquence d'origine disparaît, remplacée par la **différence** et la **somme** des deux fréquences. Ces nouvelles fréquences ne sont pas des multiples de f_x : c'est pourquoi le résultat ne « sonne » plus juste.



Spectre mesuré : sinus 1 kHz × porteuse 300 Hz sinusoïdale, Mix 100 %. Il ne reste que $1000 - 300$ et $1000 + 300$ Hz.

Avec une porteuse **carrée** ou **triangulaire**, qui contiennent les harmoniques impaires $f_c, 3f_c, 5f_c \ldots$ (d'amplitudes $1/k$ pour le carré, $1/k^2$ pour le triangle), chaque harmonique crée sa propre paire de raies $f_x \pm k f_c$: le son devient plus dense et plus agressif.

5.4.3 Implémentation

La porteuse sinusoïdale vient de l'oscillateur en quadrature (section 2.4.4), sans aucun appel à `sinf()`. Les autres formes utilisent l'accumulateur de phase et `lfoWave()`. Pour des porteuses très basses (20 Hz à 40 Hz), l'effet se rapproche d'un tremolo rapide et « bourdonnant » (section 6.1).

CHAPITRE 6

Effets de modulation

Un effet de modulation fait **varier un paramètre au cours du temps** grâce à un LFO (section 2.4.4) : le volume (tremolo), la position stéréo (autopan), la phase (phaser) ou un court retard (flanger, chorus). Le son « bouge », « respire », « tourne ».

6.1 Tremolo

Fichier : `src/effects/TremoloEffect.h`

> Le tremolo fait varier le volume de façon périodique, comme si l'on tournait très vite un bouton de volume. C'est l'effet des amplis de guitare vintage (« vibrato » chez Fender) et des pulsations rythmiques de la musique électronique.

6.1.1 Paramètres

Pot	Nom	Plage	Échelle	Rôle
1	**Vitesse**	0,5 Hz à 15 Hz (déf. 5)	log	fréquence du LFO f_m
2	**Profond**	0 % à 100 % (déf. 70)	lin.	amplitude de la variation d
3	**Forme**	Sinus, Triangle, Carré	choix	forme de la pulsation
4	**Stereo**	0 % à 100 %	lin.	déphasage gauche/droite, de 0° à 180°

6.1.2 Modèle

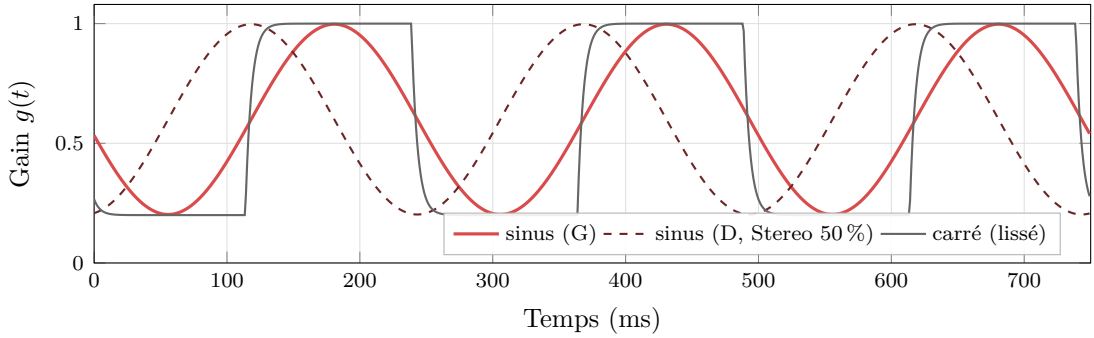
Le signal est multiplié par un gain qui oscille entre $1 - d$ et 1 :

$$y(t) = x(t)\, g(t), \qquad g(t) = 1 - d\left(\tfrac{1}{2} + \tfrac{1}{2}\, w(\varphi(t))\right).$$

Le canal droit lit le LFO avec un décalage de phase de Stereo/2 période : à 100 %, les deux canaux pulsent en alternance, et le son semble rebondir d'un côté à l'autre.

Vu en fréquence. Pour $w = \sin$ et une note de fréquence f_x, développer le produit montre que le tremolo ajoute deux raies latérales à $f_x \pm f_m$, d'amplitude $d/4$ chacune. Aux vitesses lentes, elles sont si proches de la note qu'on n'entend qu'une pulsation ; vers 15 Hz et au-delà, elles deviennent audibles comme une rugosité (c'est le principe de la modulation d'amplitude AM).

Lissage du carré. Un LFO carré ferait passer le gain instantanément de 1 à $1 - d$, ce qui crée un clic à chaque transition. Le gain est donc lissé par un filtre à un pôle de coefficient 0,995, soit une constante de temps $\tau = -1/(f_s \ln 0{,}995) \approx 4{,}5$ ms : assez court pour garder un rythme net, assez long pour supprimer les clics.



Gain mesuré en sortie du vrai effet sur un signal constant : 4 Hz, profondeur 80 %. Avec Stereo à 50 %, le canal droit est décalé d'un quart de période.

Dans le code — `TremoloEffect::process()`

```
for (int i = 0; i < n; ++i) {
  lfo_.advance();
  const float tl = 1.f - depth * (0.5f + 0.5f * lfo_.value(shape));
  const float tr = 1.f - depth * (0.5f + 0.5f * lfo_.value(shape, off));
  gL_ = tl + (gL_ - tl) * 0.995f;              // lissage (supprime les clics du carre)
  gR_ = tr + (gR_ - tr) * 0.995f;
  L[i] *= gL_;  R[i] *= gR_;
}
```

6.2 AutoPan

Fichier : `src/effects/AutoPanEffect.h`

> L'autopan déplace le son de gauche à droite et inversement, en continu. Au casque, l'effet est spectaculaire : la musique tourne autour de la tête.

6.2.1 Paramètres

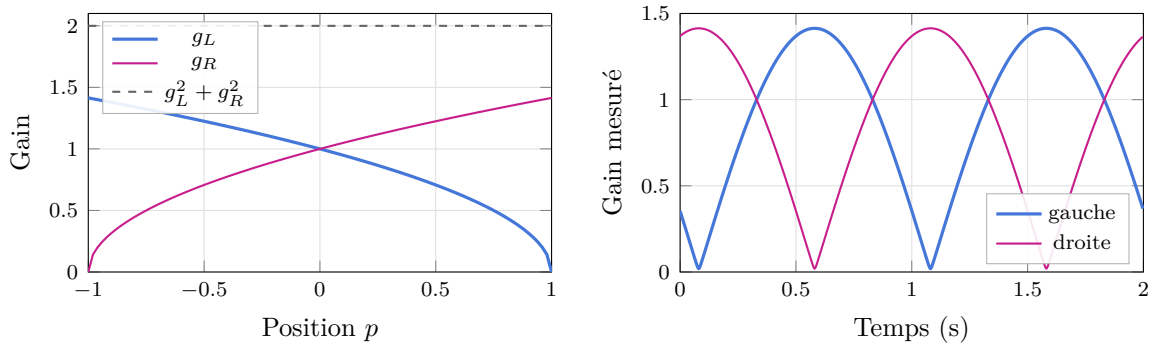
Pot	Nom	Plage	Échelle	Rôle
1	**Vitesse**	0,05 Hz à 10 Hz (déf. 0,5)	log	fréquence des allers-retours
2	**Profond**	0 % à 100 % (déf. 80)	lin.	amplitude du déplacement
3	**Forme**	Sinus, Triangle, Carré	choix	mouvement continu, linéaire ou saut

6.2.2 Modèle : panoramique à puissance constante

La position $p \in [-1, 1]$ (-1 : gauche, $+1$: droite) suit le LFO, $p = d \cdot w(\varphi)$, lissée comme pour le tremolo. Chaque canal est pondéré par

$$g_L = \sqrt{2}\sqrt{\frac{1-p}{2}}, \qquad g_R = \sqrt{2}\sqrt{\frac{1+p}{2}}, \qquad g_L^2 + g_R^2 = 2 \text{ (constant)}.$$

Le facteur $\sqrt{2}$ garantit un gain de 1 au centre ($p = 0$). Une loi *linéaire* ($g_L = 1 - p$, $g_R = 1 + p$) ferait baisser la puissance totale de 3 dB au centre : on entendrait un « trou » à chaque passage.



À gauche, la loi de panoramique. À droite, les gains mesurés sur le vrai effet (1 Hz, profondeur 100 %).

Les gains s'appliquent à chaque canal séparément, comme une balance automatique. Sur un morceau stéréo très large, un instrument placé tout à gauche ne passera donc jamais à droite : il disparaît puis revient. Pour un vrai déplacement, il faudrait d'abord sommer les canaux en mono.

6.3 Phaser

Fichier : `src/effects/PhaserEffect.h`

> Le phaser crée des « creux » dans le spectre qui se déplacent lentement : le son semble tourner, avec un effet de souffle ou de vague. Il est très utilisé sur les claviers électriques et les guitares des années 1970.

6.3.1 Paramètres

Pot	Nom	Plage	Échelle	Rôle
1	**Vitesse**	0,05 Hz à 5 Hz (déf. 0,4)	log	vitesse du balayage
2	**Profond**	0 % à 100 % (déf. 85)	lin.	étendue du balayage
3	**Feedbk**	0 % à 90 % (déf. 50)	lin.	réinjection : creux plus marqués
4	**Etages**	4, 6, 8 ou 12 (déf. 6)	choix	nombre de passe-tout, donc de creux

6.3.2 Le filtre passe-tout

Un **passe-tout** du premier ordre a un gain de 1 à toutes les fréquences mais *décale la phase*, et ce décalage dépend de la fréquence :

$$A(z) = \frac{a + z^{-1}}{1 + a\,z^{-1}}, \qquad |A(\mathrm{e}^{\mathrm{j}\omega})| = 1, \qquad a = \frac{\tan(\pi f_c/f_s) - 1}{\tan(\pi f_c/f_s) + 1}.$$

Le déphasage vaut $\approx -2\arctan(f/f_c)$: 0° pour les graves, −90° à f_c, et il tend vers −180° dans les aigus. En plaçant N passe-tout en série, le déphasage total va jusqu'à $-N \times 180°$.

6.3.3 Les creux

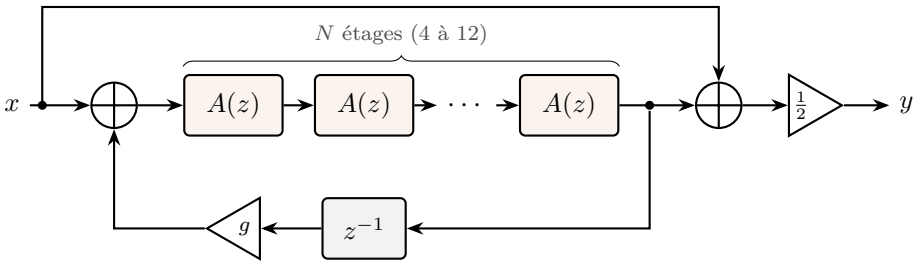
La sortie est la **moyenne** du signal direct et du signal déphasé :

$$y = \tfrac{1}{2}\,(x + A^N x).$$

Aux fréquences où le déphasage total est un multiple impair de 180°, les deux signaux sont en opposition et s'annulent : c'est un **creux** (*notch*). Avec un déphasage de $-2N\arctan(f/f_c)$, les creux tombent en

$$f_k = f_c\,\tan\!\left(\frac{(2k+1)\,\pi}{2N}\right), \qquad k = 0, 1, \ldots, \tfrac{N}{2} - 1,$$

soit $N/2$ creux. Pour $N = 6$: 0,27 f_c, f_c et 3,7 f_c.



Réinjection. Une partie g de la sortie de la chaîne est renvoyée à l'entrée (avec un retard d'un échantillon pour que le calcul soit possible). La fonction de transfert devient

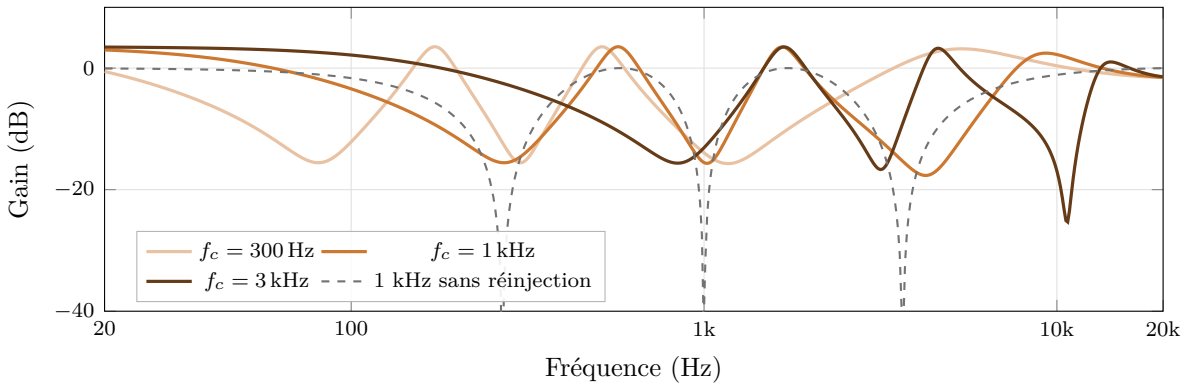
$$H(z) = \frac{1}{2}\left(1 + \frac{A(z)^N}{1 - g\,z^{-1}A(z)^N}\right) :$$

les creux se creusent et des bosses résonantes apparaissent entre eux.

Balayage. La fréquence f_c de tous les étages suit le LFO $m \in [-1, 1]$ en échelle logarithmique :

$$f_c = 200\,\text{Hz} \cdot 2^{\left(\frac{1}{2}+\frac{1}{2}m\right)\cdot d\cdot 4{,}3},$$

soit de 200 Hz à environ 3,9 kHz à profondeur maximale. Le canal droit utilise le cosinus de l'oscillateur en quadrature : ses creux sont décalés d'un quart de période, ce qui donne l'impression d'un tourbillon dans l'espace stéréo.



Réponses à 6 étages et 50 % de réinjection, à trois positions du balayage. Les trois creux se déplacent ensemble.

6.3.4 Implémentation

Dans le code — `PhaserEffect::process()` (extrait)

```
if ((i & 7) == 0) {                          // coefficients tous les 8 echantillons
  aL = apCoef(sweepFreq(osc_.s));            // canal gauche : sinus
  aR = apCoef(sweepFreq(osc_.c));            // canal droit : cosinus (+90 deg)
}
float l = L[i] + fb_[0] * fb;                // reinjection
for (int s = 0; s < stages; ++s) {           // chaine de passe-tout
  const float yl = aL * l + z_[0][s];
  z_[0][s] = l - aL * yl;
  l = yl;
}
fb_[0] = dsp::softClip(l);
L[i] = 0.5f * (L[i] + l);                    // somme avec le signal direct
```

— **Approximation de la tangente.** $\tan x \approx x + x^3/3 + 2x^5/15$ (développement de Taylor). Pour $f_c = 4\,\text{kHz}$, $x = 0{,}285$ et l'erreur relative est inférieure à 10^{-4}.

— **Mise à jour tous les 8 échantillons**, soit 5,5 kHz : largement assez pour un balayage de quelques hertz, et cela divise par 8 le coût du calcul des coefficients.

— **Coût.** Avec 12 étages : 24 multiplications par échantillon et par canal.

6.4 Flanger

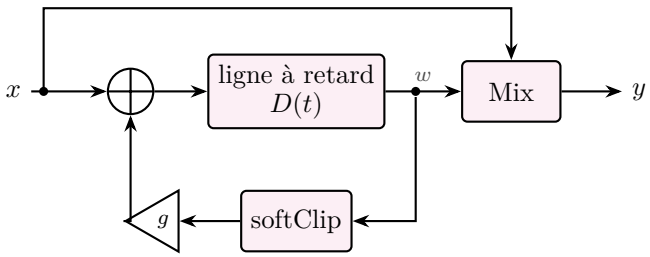
Fichier : `src/effects/FlangerEffect.h`

> Le flanger mélange le son avec une copie de lui-même retardée de quelques millisecondes, retard qui varie lentement. Le résultat est un « avion à réaction » qui passe, un sifflement métallique qui monte et descend. Le nom vient de la technique d'origine : on freinait du doigt le bord (*flange*) de la bobine d'un des deux magnétophones qui jouaient la même bande.

6.4.1 Paramètres

Pot	Nom	Plage	Échelle	Rôle
1	**Vitesse**	0,05 Hz à 5 Hz (déf. 0,25)	log	vitesse du balayage
2	**Profond**	0 % à 100 % (déf. 70)	lin.	excursion du retard
3	**Feedbk**	−95 à +95 % (déf. +60)	lin.	réinjection : résonance métallique
4	**Mix**	0 % à 100 % (déf. 50)	lin.	dosage

6.4.2 Modèle



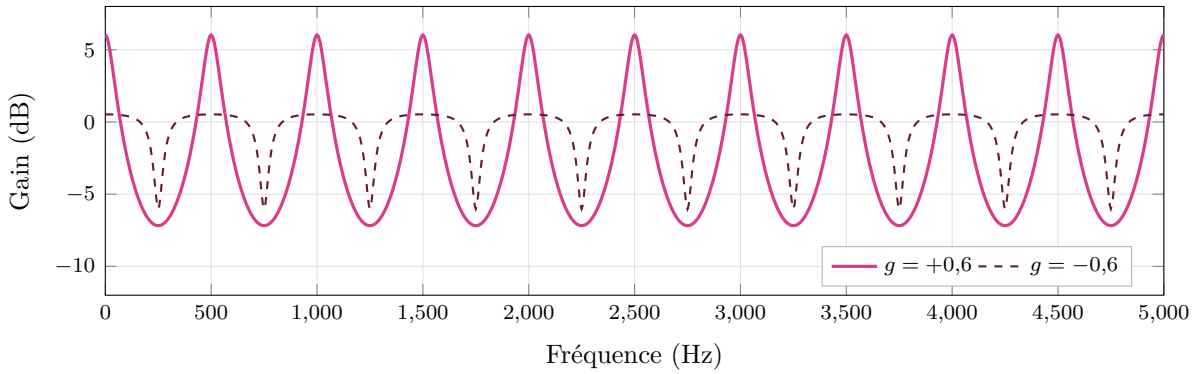
Le retard oscille entre 0,3 ms et $0{,}3\,\text{ms} + d \cdot 4{,}5\,\text{ms}$:

$$D(t) = 0{,}3\,\text{ms} + d \cdot 4{,}5\,\text{ms} \cdot \left(\tfrac{1}{2} + \tfrac{1}{2}\sin 2\pi f_m t\right).$$

En figeant le retard à une valeur D (en échantillons), le signal retardé avec réinjection vaut $W(z) = \dfrac{z^{-D}}{1 - g\,z^{-D}}\,X(z)$ et la sortie

$$H(z) = \left(1 - \frac{m}{2}\right) + m\,\frac{z^{-D}}{1 - g\,z^{-D}}.$$

C'est un **filtre en peigne**. Pour $g > 0$, les pics tombent aux multiples de $1/D$ et les creux à mi-chemin ; pour $g < 0$, les pics se placent aux multiples impairs de $1/(2D)$, ce qui donne une couleur plus creuse. Plus $|g|$ est proche de 1, plus les pics sont étroits et hauts. Quand D varie, toutes les dents du peigne se déplacent ensemble, d'où le « whoosh ».



Peigne obtenu pour un retard figé $D = 2\,\text{ms}$ (pics tous les 500 Hz), Mix 50 %.

6.4.3 Implémentation

— Le retard varie en continu : la lecture utilise l'**interpolation linéaire** (section 2.4.3). Sans elle, le retard sauterait d'échantillon en échantillon et produirait un grésillement.

— Le canal droit suit le cosinus de l'oscillateur (90° de décalage) : les peignes gauche et droit ne coïncident jamais, ce qui élargit l'image stéréo.

— La réinjection passe par `softClip` : même à ±95 % et avec un signal fort, la boucle ne peut pas s'emballer.

— Tampon de 10 ms par canal (480 échantillons à 48 kHz), soit ≈4 ko au total.

6.5 Chorus

Fichier : `src/effects/ChorusEffect.h`

> Le chorus simule plusieurs musiciens qui jouent la même partie : jamais parfaitement ensemble, jamais parfaitement accordés. Le son s'épaissit, devient chatoyant et s'élargit dans l'espace.

6.5.1 Paramètres

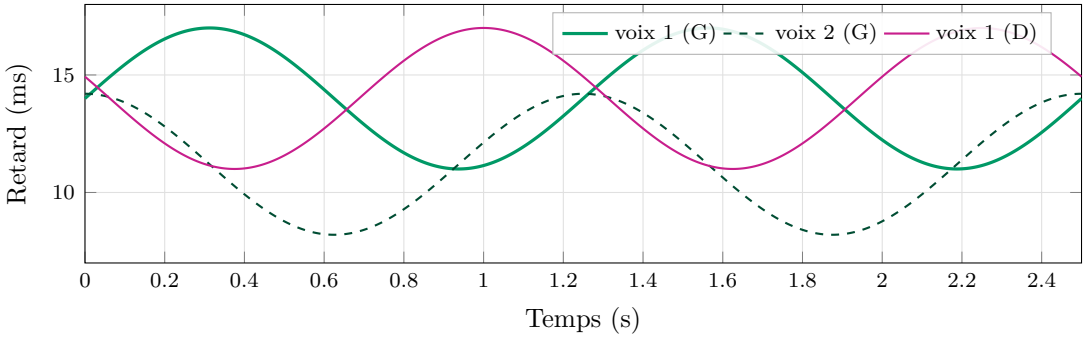
Pot	Nom	Plage	Échelle	Rôle
1	**Vitesse**	0,05 Hz à 5 Hz (déf. 0,8)	log	vitesse de l'ondulation
2	**Profond**	0 % à 100 % (déf. 50)	lin.	excursion du retard (jusqu'à ±6 ms)
3	**Largeur**	0 % à 100 % (déf. 80)	lin.	décalage gauche/droite (90° à 180°)
4	**Mix**	0 % à 100 % (déf. 50)	lin.	dosage

6.5.2 Modèle

Le chorus utilise le même principe que le flanger, avec des retards plus longs (10 ms à 20 ms) et sans réinjection : on n'entend plus un filtre en peigne mais une *copie* légèrement désaccordée. Chaque canal comporte **deux voix** lues dans la même ligne à retard, pilotées par le sinus et le cosinus de l'oscillateur en quadrature :

$$D_1(t) = 14\,\text{ms} + A\sin\theta(t), \qquad D_2(t) = 11{,}2\,\text{ms} + A\cos\theta(t), \qquad A = d\cdot 6\,\text{ms},$$
$$w_L = \tfrac{1}{2}\big[x(t-D_1) + x(t-D_2)\big], \qquad y_L = (1-\tfrac{m}{2})x_L + m\,w_L.$$

Le canal droit utilise le même calcul avec une phase décalée de $\phi = \pi\,(\frac{1}{2} + \frac{1}{2}\,\text{Largeur})$.



Retards des voix avec les réglages par défaut (0,8 Hz, profondeur 50 %, largeur 80 %).

6.5.3 D'où vient le désaccord ? L'effet Doppler

Lire un tampon avec un retard qui change revient à lire le son plus vite ou plus lentement que son enregistrement. Le rapport de vitesse de lecture vaut

$$r(t) = 1 - \frac{\mathrm{d}D}{\mathrm{d}t},$$

et il modifie la hauteur, exactement comme une sirène d'ambulance qui s'approche ou s'éloigne. Pour $D = A\sin(2\pi f t)$, la variation maximale est $2\pi f A$. Avec les réglages par défaut ($A = 3$ ms, $f = 0{,}8$ Hz) :

$$\Delta_{\max} = 1200\log_2(1 + 2\pi \cdot 0{,}8 \cdot 0{,}003) \approx \pm 26 \text{ cents},$$

soit environ un quart de demi-ton : assez pour l'effet d'ensemble, pas assez pour sonner faux. À vitesse et profondeur maximales, on atteint plusieurs demi-tons : l'effet devient un vibrato désordonné.

Dans le code — `ChorusEffect::process()` (extrait)

```
osc_.step();
const float s = osc_.s, c = osc_.c;          // voix 1 : sin, voix 2 : cos
line_[0].write(L[i]);
const float wl = 0.5f * (line_[0].readFrac(base + depth * s) +
                         line_[0].readFrac(base * 0.8f + depth * c));
const float sr = osc_.sinOffset(co, so);     // dephasage du canal droit
...
L[i] = L[i] * dry + wl * mix;
```

Coût et mémoire. Quatre lectures interpolées par trame. Tampon de 40 ms par canal (1 920 échantillons à 48 kHz), soit 15 ko en PSRAM.

CHAPITRE 7

Effets temporels

Les effets temporels reproduisent l'**espace** : des échos distincts (delay) ou la multitude de réflexions d'une pièce (reverb). Ils utilisent de longues lignes à retard, placées en PSRAM.

7.1 Delay

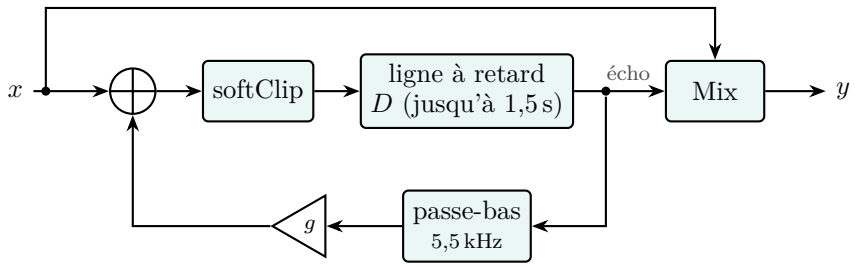
Fichier : `src/effects/DelayEffect.h`

> Le delay répète le son après un temps réglable, de plus en plus faiblement : l'écho de la montagne. En mode *ping-pong*, les répétitions rebondissent d'un haut-parleur à l'autre.

7.1.1 Paramètres

Pot	Nom	Plage	Échelle	Rôle
1	**Temps**	20 ms à 1 500 ms (déf. 350)	log	intervalle entre deux répétitions
2	**Feedbk**	0 % à 95 % (déf. 45)	lin.	part réinjectée : nombre de répétitions
3	**Mix**	0 % à 100 % (déf. 35)	lin.	niveau des échos
4	**Mode**	Stéréo ou Ping-pong (déf.)	choix	répartition des échos

7.1.2 Schéma-bloc



7.1.3 Modèle

En mode stéréo, chaque canal possède sa ligne ℓ :

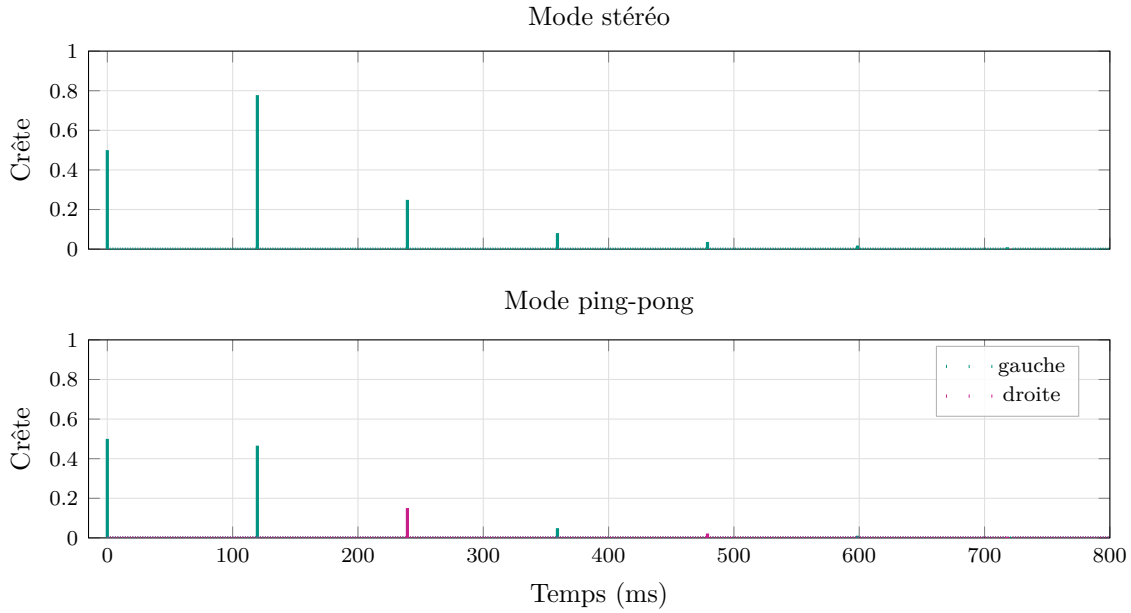
$$\begin{aligned} e[n] &= \ell[n - D] \quad \text{(lecture interpolée)},\\ \ell[n] &= \mathrm{softClip}\big(x[n] + g \cdot \mathrm{LP}(e[n])\big),\\ y[n] &= \big(1 - \tfrac{m}{2}\big)x[n] + m\, e[n]. \end{aligned}$$

En ignorant le passe-bas et la saturation, la réponse à une impulsion est une suite d'échos aux instants $D, 2D, 3D, \ldots$ d'amplitudes $m, mg, mg^2, \ldots$ Le nombre d'échos audibles (au-dessus de $-60\,\mathrm{dB}$) vaut environ $\ln(10^{-3})/\ln g$: 9 échos pour $g = 0{,}45$, 135 pour $g = 0{,}95$.

Mode ping-pong. La ligne gauche reçoit l'entrée sommée en mono et les échos de *droite* ; la ligne droite ne reçoit que les échos de *gauche* :

$$\ell_L[n] = \mathrm{softClip}\big(\tfrac{1}{2}(x_L + x_R) + g \cdot \mathrm{LP}(e_R)\big), \qquad \ell_R[n] = \mathrm{softClip}\big(g \cdot \mathrm{LP}(e_L)\big).$$

Le premier écho sort à gauche, le deuxième à droite, le troisième à gauche, et ainsi de suite.



Réponses impulsionnelles mesurées sur le vrai code (temps 120 ms, feedback 60 %, mix 100 %), en crête par fenêtre de 2 ms. L'impulsion à $t = 0$ est le son direct. Les échos diminuent plus vite que g^k : le passe-bas de la boucle étale l'impulsion à chaque passage.

7.1.4 Détails d'implémentation

Glissement « bande magnétique ». Quand on tourne le potentiomètre **Temps**, le retard courant rejoint la cible par un lissage à un pôle appliqué à chaque échantillon :

$$D \leftarrow D + 0{,}0005\,(D_{\text{cible}} - D), \qquad \tau = \frac{1}{0{,}0005\,f_s} \approx 45\,\text{ms}.$$

Le retard variant en continu, la hauteur des échos change pendant la transition (effet Doppler, voir le chorus), exactement comme sur un écho à bande dont on change la vitesse. C'est plus musical qu'un saut brutal, qui produirait un clic.

Passe-bas dans la boucle. Chaque répétition traverse un passe-bas à un pôle à 5,5 kHz. Les échos s'assombrissent progressivement, comme sur les delays analogiques et à bande, et les répétitions se fondent dans le mix au lieu de le saturer d'aigus.

Stabilité. Le `softClip` à l'entrée de la ligne garantit que, même avec 95 % de réinjection et une entrée forte, la boucle ne peut pas croître sans limite : elle sature doucement.

Mémoire. Les lignes sont dimensionnées pour 1,5 s à 48 kHz, soit 72 008 échantillons par canal : 2×288 ko $= 576$ ko de PSRAM. À l'activation, `reset()` efface les deux lignes, ce qui prend quelques millisecondes ; les 46 ms de tampons DMA I2S absorbent cette pause.

Dans le code — `DelayEffect::process()`

```
for (int i = 0; i < n; ++i) {
  d += (target - d) * 0.0005f;                    // glissement facon "bande"
  const float yl = line_[0].readFrac(d), yr = line_[1].readFrac(d);
  const float fl = lp_[0].process(yl), fr = lp_[1].process(yr);
  if (pp) {                                       // ping-pong : croisement des echos
    line_[0].write(dsp::softClip(0.5f * (L[i] + R[i]) + fb * fr));
    line_[1].write(dsp::softClip(fb * fl));
  } else {
    line_[0].write(dsp::softClip(L[i] + fb * fl));
```

```
    line_[1].write(dsp::softClip(R[i] + fb * fr));
  }
  L[i] = L[i] * dry + yl * mix;
  R[i] = R[i] * dry + yr * mix;
}
```

Visualisation. La page représente les répétitions par des barres de hauteur g^k (alternées en haut et en bas en mode ping-pong). Une impulsion lumineuse parcourt la ligne au rythme réel du delay.

7.2 Reverb

Fichier : `src/effects/ReverbEffect.h`

> Dans une pièce, le son rebondit sur les murs, le plafond, les meubles : on entend des milliers de réflexions de plus en plus nombreuses et faibles, qui se fondent en une « traîne ». La reverb recrée cette traîne : petite salle, grande église, cathédrale.

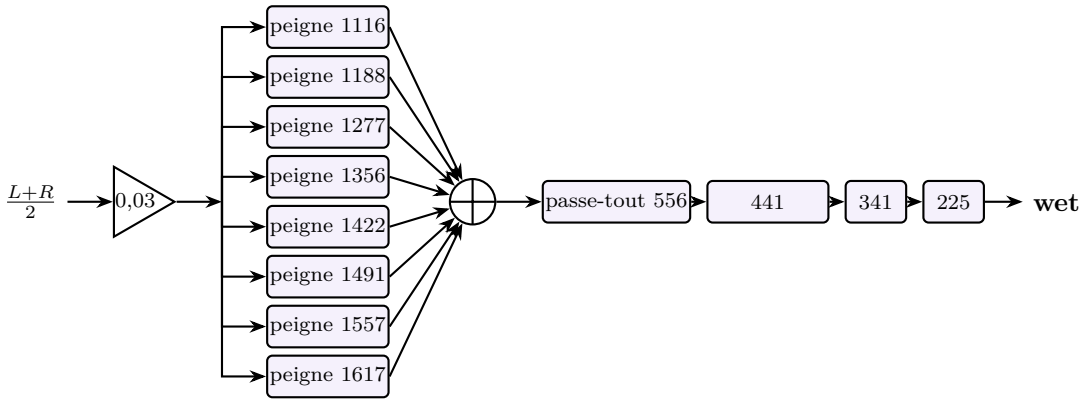
7.2.1 Paramètres

Pot	Nom	Plage	Échelle	Rôle
1	**Taille**	0 % à 100 % (déf. 75)	lin.	durée de la traîne
2	**Amorti**	0 % à 100 % (déf. 45)	lin.	absorption des aigus
3	**Largeur**	0 % à 100 % (déf. 100)	lin.	largeur stéréo de la traîne
4	**Mix**	0 % à 100 % (déf. 30)	lin.	niveau de réverbération

7.2.2 L'algorithme Freeverb

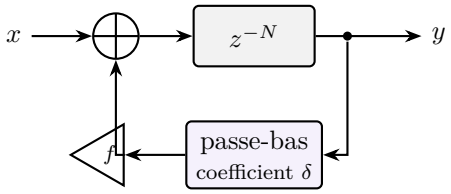
L'effet reprend l'algorithme *Freeverb* (Jezar, 2000), descendant des réverbérations de Schroeder et Moorer. Par canal, il comporte :

1. **8 filtres en peigne amortis en parallèle** (LBCF, *lowpass-feedback comb filters*) : chacun produit une suite d'échos réguliers qui décroît ; leurs longueurs différentes entrelacent les échos et donnent une densité élevée ;
2. **4 filtres « passe-tout » en série**, qui diffusent les échos : chaque impulsion devient un petit nuage d'impulsions, et la traîne devient lisse.



Un canal (longueurs en échantillons à 44,1 kHz). Le canal droit ajoute 23 échantillons à chaque longueur pour décorréler les deux traînes. Le gain d'entrée réel est $0{,}015 \times (L + R) = 0{,}03 \times \frac{L+R}{2}$.

7.2.3 Le peigne amorti



Le peigne retarde le signal de N échantillons et en réinjecte une fraction f à travers un passe-bas à un pôle :

$$y[n] = v[n-N], \qquad v[n] = x[n] + f\,s[n],$$
$$s[n] = (1-\delta)\,y[n] + \delta\,s[n-1] \quad \text{(passe-bas de la boucle)},$$

avec $f = 0{,}7 + 0{,}28 \cdot \text{Taille}$ et $\delta = 0{,}4 \cdot \text{Amorti}$. Sans amortissement, $H(z) = z^{-N}/(1 - fz^{-N})$: un écho tous les N échantillons, multiplié par f à chaque tour.

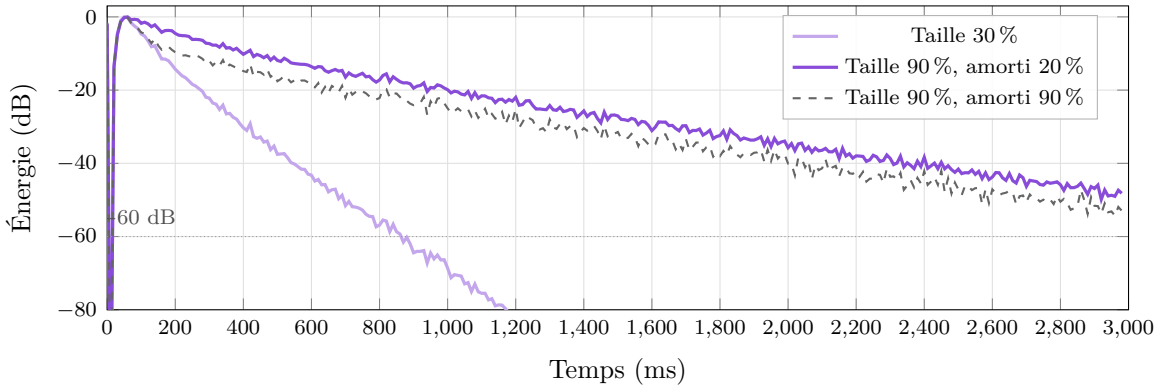
Durée de réverbération. Le temps T_{60} nécessaire pour que la traîne baisse de 60 dB se déduit du nombre de tours k tel que $f^k = 10^{-3}$, chaque tour durant N/f_s :

$$T_{60} = \frac{-3\,N}{f_s\,\log_{10} f}.$$

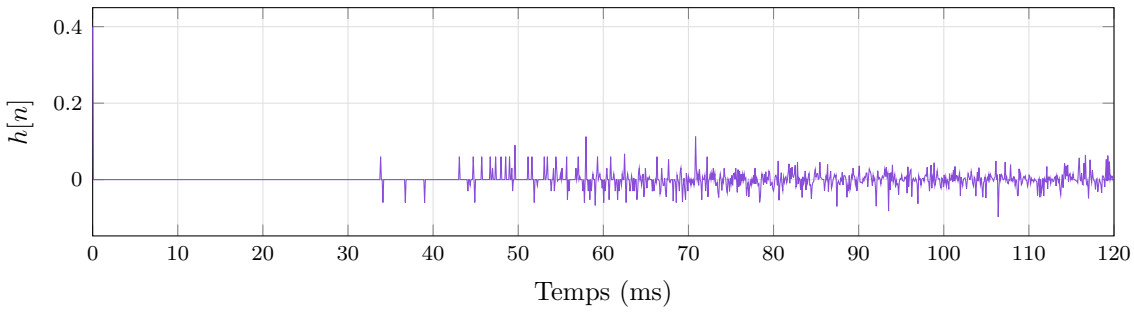
Pour la longueur moyenne $\bar{N} \approx 1378$:

Taille	f	T_{60} théorique	T_{60} mesuré (code réel)
30 %	0,784	0,89 s	$\approx$ 0,9 s
75 %	0,910	2,3 s	—
90 %	0,952	4,4 s	$\approx$ 3,7 s (amorti 20 %)

L'**amortissement** réduit le gain de boucle dans les aigus : ils s'éteignent plus vite que les graves, comme dans une vraie salle où l'air, les rideaux et le public absorbent surtout les hautes fréquences. Le T_{60} global mesuré est donc un peu plus court que le calcul.



Décroissance d'énergie de la réponse impulsionnelle réelle, par fenêtres de 10 ms. Une traîne exponentielle donne une droite en décibels.



Début de la réponse impulsionnelle (Taille 90 %). Les premiers échos apparaissent après ≈25 ms (le plus court des peignes) puis se densifient très vite.

7.2.4 Le diffuseur

Le « passe-tout » de Freeverb s'écrit

$$y[n] = b[n] - x[n], \qquad b[n] = v[n-N], \qquad v[n] = x[n] + 0{,}5\, b[n],$$

soit $H(z) = \dfrac{-1 + 1{,}5\, z^{-N}}{1 - 0{,}5\, z^{-N}}$. Ce n'est pas exactement un passe-tout (celui de Schroeder serait $\frac{-g+z^{-N}}{1-gz^{-N}}$), mais c'est la variante de l'algorithme d'origine. Son rôle est de « brouiller » la structure régulière des échos des peignes, qui sonnerait sinon métallique.

7.2.5 Mélange stéréo

Les sorties des deux canaux sont croisées selon la largeur w et le mix m :

$$\text{wet}_1 = 2m\left(\tfrac{w}{2} + \tfrac{1}{2}\right), \qquad \text{wet}_2 = 2m\,\tfrac{1-w}{2}, \qquad \text{dry} = 1 - 0{,}6\, m,$$
$$L' = \text{dry}\cdot L + \text{wet}_1\, r_L + \text{wet}_2\, r_R, \qquad R' = \text{dry}\cdot R + \text{wet}_1\, r_R + \text{wet}_2\, r_L.$$

À $w = 0$, les deux canaux reçoivent la même traîne (mono) ; à $w = 1$, chacun reçoit la sienne, et les deux traînes, décorrélées par les 23 échantillons d'écart, enveloppent l'auditeur.

7.2.6 Implémentation

Dans le code — `ReverbEffect::process()` et `dsp::Comb`

```
inline float Comb::process(float x, float feedback, float damp) {
  const float out = buf[pos];
  store = out * (1.f - damp) + store * damp;     // passe-bas dans la boucle
  buf[pos] = x + store * feedback;
  if (++pos >= size) pos = 0;
  return out;
}
...
const float in = (L[i] + R[i]) * 0.015f;
float ol = 0, orr = 0;
for (int c = 0; c < NC; ++c) {                   // 8 peignes en parallele
  ol += comb_[0][c].process(in, roomFb_, damp_);
  orr += comb_[1][c].process(in, roomFb_, damp_);
}
for (int a = 0; a < NA; ++a) {                   // 4 diffuseurs en serie
  ol = allp_[0][a].process(ol);
  orr = allp_[1][a].process(orr);
}
L[i] = L[i] * dry_ + ol * wet1_ + orr * wet2_;
R[i] = R[i] * dry_ + orr * wet1_ + ol * wet2_;
```

— **Adaptation à f_s.** Les longueurs d'origine sont prévues pour 44,1 kHz. Elles sont multipliées par $f_s/44100$ dans `prepare()`, pour que la pièce garde la même taille à 48 kHz.

— **Mémoire.** Un seul bloc de 28 024 flottants (112 ko) en PSRAM, découpé entre les 24 tampons.

— **Coût.** 24 lectures/écritures en PSRAM par trame : c'est l'effet le plus gourmand, de l'ordre de quelques pourcents du processeur. La PSRAM est lente en accès aléatoire, mais chaque tampon est parcouru séquentiellement, et le cache de l'ESP32 limite les pertes.

— **Visualisation.** La page dessine une réponse impulsionnelle stylisée (réflexions précoces puis décroissance exponentielle), dont la longueur suit la taille et dont la couleur se refroidit avec l'amortissement.

CHAPITRE 8

Sortie et analyse

8.1 Master

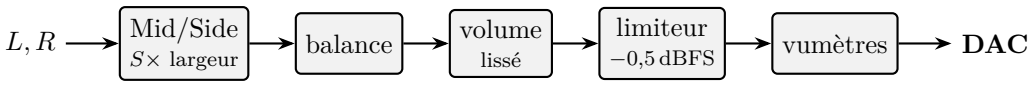
Fichier : `src/effects/MasterEffect.h`

Le Master est un effet comme les autres (il hérite de `Effect`) mais il est **toujours actif** (`canBypass()` renvoie `false`) et toujours placé **en fin de chaîne**, après tous les effets réordonnables. Il se règle depuis le menu *Master* ; sur les autres écrans, le potentiomètre 1 pilote directement son volume.

8.1.1 Paramètres

Pot	Nom	Plage	Échelle	Rôle
1	**Volume**	0 % à 150 % (déf. 80)	lin.	gain général
2	**Balance**	−1 (gauche) à +1 (droite)	lin.	équilibre gauche/droite
3	**Largeur**	0 % à 200 % (déf. 100)	lin.	largeur stéréo (Mid/Side)
4	**Limiteur**	Off / On (déf. On)	choix	protection contre l'écrêtage

8.1.2 Chaîne



$$M = \tfrac{1}{2}(L+R), \quad S = \tfrac{1}{2}(L-R)\cdot w, \qquad g_L = \begin{cases} 1-b & b>0 \\ 1 & \text{sinon} \end{cases}, \quad g_R = \begin{cases} 1+b & b<0 \\ 1 & \text{sinon} \end{cases},$$

$$L' = (M+S)\,g_L\,v, \qquad R' = (M-S)\,g_R\,v.$$

Volume lissé. Le gain v rejoint la valeur du potentiomètre avec un coefficient 0,002 par échantillon ($\tau \approx 11$ ms). Tourner le bouton ne produit ni craquement ni « escalier » audible.

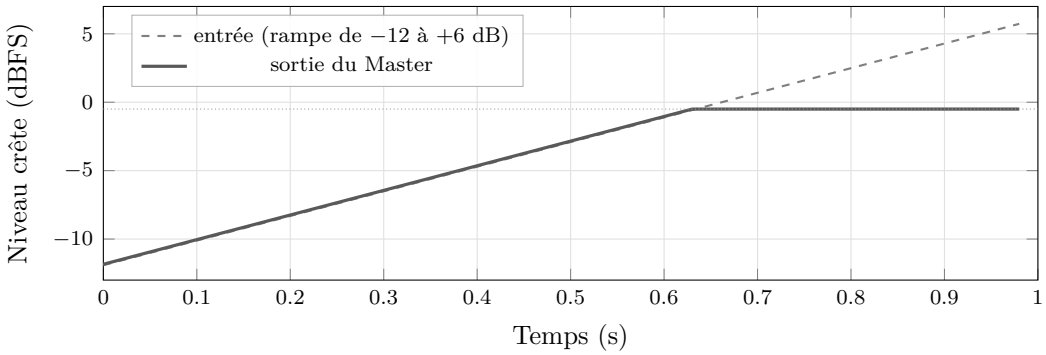
Volume du téléphone. Le volume réglé sur le téléphone (AVRCP « volume absolu ») est appliqué par la librairie Bluetooth *avant* la chaîne d'effets. Il agit donc aussi sur le niveau qui attaque la distorsion et le compresseur, comme un gain d'entrée.

8.1.3 Le limiteur

Sans protection, un signal qui dépasse ±1 est écrêté brutalement à la conversion vers le DAC, ce qui produit une distorsion dure et désagréable. Les effets peuvent facilement y conduire (égaliseur poussé, reverb à fond, volume à 150 %). Le limiteur est un compresseur extrême à attaque instantanée :

$$p = \max(|L'|, |R'|), \qquad \text{besoin} = \begin{cases} 0{,}944/p & p > 0{,}944 \\ 1 & \text{sinon} \end{cases}, \qquad e \leftarrow \begin{cases} \text{besoin} & \text{si besoin} < e \\ 1-(1-e)\left(1-\frac{1}{0{,}15\,f_s}\right) & \text{sinon.} \end{cases}$$

Le plafond 0,944 correspond à −0,5 dBFS. Le gain e descend immédiatement autant que nécessaire, puis remonte vers 1 avec une constante de temps de 150 ms.



Mesure sur le vrai code : la sortie suit l'entrée jusqu'à $-0{,}5$ dBFS, puis reste plafonnée même quand l'entrée dépasse la pleine échelle de 6 dB.

Attention

Le limiteur n'a pas d'anticipation (*look-ahead*). Une attaque instantanée sur un échantillon isolé modifie brutalement le gain, ce qui produit une légère distorsion sur les transitoires très violents. C'est un filet de sécurité, pas un outil de mastering : si le témoin de crête reste collé au rouge, il vaut mieux baisser le volume.

Vumètres. Pour chaque bloc, on garde la crête de chaque canal, avec une retombée par bloc niveau $\leftarrow \max(\text{crête}, 0{,}93 \cdot \text{niveau})$, soit une constante de temps d'environ 40 ms. La page du Master affiche deux barres en dB (de -48 à 0 dB, vertes, puis orange et rouges), un repère de crête maintenue, et une ellipse dont la largeur suit le réglage **Largeur** et dont la position suit la balance.

8.2 L'analyseur de spectre

Fichiers : `src/ui/Spectrum.cpp`, `src/dsp/Fft.h`

L'écran d'accueil affiche le spectre du signal **de sortie** (après tous les effets et le Master), qui montre en direct l'action des filtres et des effets. La page de chaque effet le reprend en transparence derrière sa visualisation.

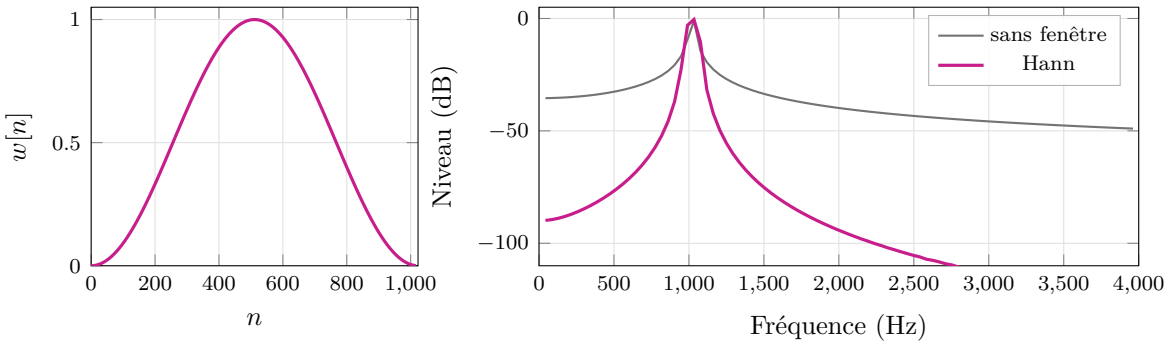
8.2.1 Capture

La tâche audio recopie chaque échantillon de sortie, sommé en mono ($\frac{L+R}{2}$), dans un tampon circulaire de $N = 1024$ cases. À chaque image, la tâche d'interface en copie le contenu dans l'ordre chronologique. La fenêtre analysée dure $N/f_s = 23$ ms et la résolution vaut $\Delta f = f_s/N \approx 43$ Hz.

8.2.2 Fenêtre de Hann

La FFT suppose que le morceau analysé se répète à l'infini. Si le signal ne contient pas un nombre entier de périodes, la « couture » crée une discontinuité qui étale l'énergie sur tout le spectre : c'est la **fuite spectrale**. On multiplie donc le bloc par une fenêtre qui s'annule aux deux extrémités :

$$w[n] = \frac{1}{2} - \frac{1}{2}\cos\frac{2\pi n}{N-1}.$$



À droite : spectre d'un sinus à 1 022 Hz, placé entre deux raies de la FFT (cas le plus défavorable), calculé avec `dsp::Fft`. Sans fenêtre, la fuite reste vers −45 dB jusqu'à plusieurs kilohertz ; avec la fenêtre de Hann, elle passe sous −75 dB dès 1,5 kHz et sous −110 dB à 3 kHz.

8.2.3 La FFT radix-2

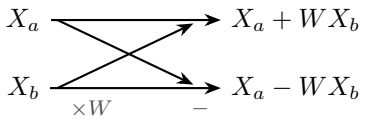
La transformée de Fourier discrète

$$X_k = \sum_{n=0}^{N-1} x[n]\, \mathrm{e}^{-2\pi \mathrm{j}\, kn/N}$$

coûterait $N^2 \approx 10^6$ multiplications complexes. L'algorithme de Cooley-Tukey (*FFT*) sépare récursivement les échantillons pairs et impairs et réduit le coût à $\frac{N}{2}\log_2 N = 5120$ « papillons » :

$$\begin{aligned} X'_a &= X_a + W\, X_b, \\ X'_b &= X_a - W\, X_b, \end{aligned} \qquad W = \mathrm{e}^{-2\pi \mathrm{j}\, k/\ell},$$

où ℓ est la taille du sous-problème (2, 4, …, 1024). L'implémentation est itérative : une permutation par **inversion des bits** des indices range d'abord les échantillons dans l'ordre voulu, puis $\log_2 N = 10$ passes de papillons sont effectuées en place. Les facteurs W (les *twiddles*) sont précalculés dans des tables de cosinus et de sinus.

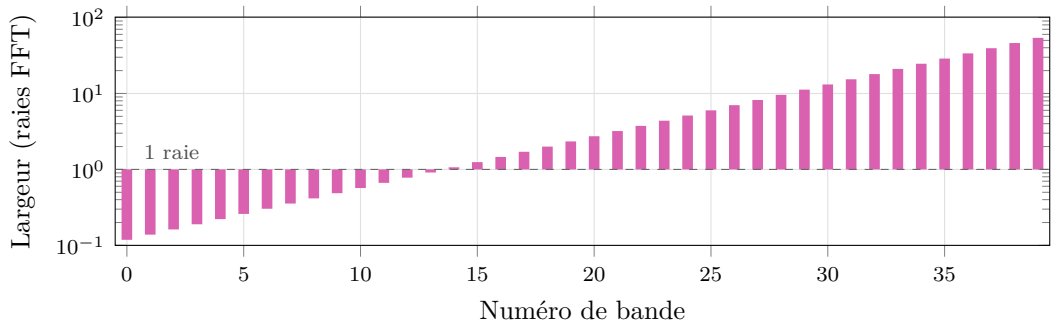


Un papillon : deux entrées, deux sorties, une multiplication complexe.

Normalisation. Pour qu'une sinusoïde pleine échelle s'affiche à 0 dB : $\times\frac{2}{N}$ car le spectre réel est symétrique et on n'en affiche qu'une moitié, et $\times 2$ pour compenser le gain moyen $\frac{1}{2}$ de la fenêtre de Hann, soit $|X_k| \cdot \frac{4}{N}$.

8.2.4 Des raies aux bandes affichées

L'écran ne fait que 160 pixels de large. Les 512 raies sont regroupées en **40 bandes logarithmiques** de 30 Hz à 16 kHz. Chaque bande couvre un rapport de fréquences de $(16000/30)^{1/40} \approx 1{,}17$, soit environ un quart d'octave : l'affichage suit la perception de l'oreille, et chaque octave occupe la même largeur à l'écran.



Dans les graves, les bandes sont plus étroites qu'une raie de la FFT : leur valeur est alors *interpolée* entre les deux raies voisines. Dans les aigus, une bande regroupe des dizaines de raies, et l'on garde la plus forte.

Pente de compensation. La musique a naturellement plus d'énergie dans les graves (son spectre décroît d'environ 3 dB par octave, comme un bruit rose). Pour obtenir un affichage équilibré, on ajoute $3\log_2(f/1\,\text{kHz})$ dB à chaque bande : un bruit rose apparaît alors plat.

Mise à l'échelle et dynamique de l'affichage.

- hauteur normalisée $v = \text{clamp}\big((G_{\text{dB}} + 80)/70,\ 0,\ 1\big)$: l'affichage couvre -80 dB à -10 dB ;
- montée rapide ($v \leftarrow v + 0{,}7\,(v_{\text{mesuré}} - v)$) et descente limitée à 0,9 hauteur d'écran par seconde : les barres réagissent aux coups de batterie mais retombent doucement, sans scintiller ;
- crêtes maintenues 0,5 s, puis chute de plus en plus rapide, comme un objet qui tombe ;
- style *Courbe* : interpolation de Catmull-Rom entre les 40 bandes, pour une courbe lisse passant par chaque valeur :

$$v(t) = v_1 + \tfrac{t}{2}\Big(v_2 - v_0 + t\big(2v_0 - 5v_1 + 4v_2 - v_3 + t\,(3(v_1 - v_2) + v_3 - v_0)\big)\Big);$$

- style *Cascade* : les 112 derniers spectres défilent verticalement, colorés selon une palette « chaleur » (noir, violet, magenta, orange, jaune, blanc).

Coût. Une FFT de 1024 points par image (≈30 par seconde) représente une petite fraction du temps de calcul de l'interface. Le dessin et l'envoi de l'image à l'écran par le bus SPI (40 ko à 27 MHz, soit ≈12 ms) coûtent bien davantage.

ANNEXE A

Récapitulatif des effets

A.1 Paramètres

Effet	Pot 1	Pot 2	Pot 3	Pot 4
Low Pass	Cutoff 30 Hz–18 kHz	Reso Q 0,5–10	Pente 12/24 dB	LFO 0–8 Hz
High Pass	Cutoff 30 Hz–18 kHz	Reso Q 0,5–10	Pente 12/24 dB	LFO 0–8 Hz
Band Pass	Cutoff 30 Hz–18 kHz	Reso Q 0,5–10	Pente 12/24 dB	LFO 0–8 Hz
EQ 3 bandes	Grave ±15 dB	Médium ±15 dB	Freq 200 Hz–5 kHz	Aigu ±15 dB
Compressor	Seuil −40–0 dB	Ratio 1–20 :1	Attaque 0,1–100 ms	Relâche 10–1000 ms
Distortion	Drive 0–40 dB	Tone 0,8–12 kHz	Type (4)	Mix
Bitcrusher	Bits 1–16	Réduction ×1–×32	Mix	—
Ring Mod	Freq 20 Hz–3 kHz	Forme (3)	Mix	—
Tremolo	Vitesse 0,5–15 Hz	Profondeur	Forme (3)	Stéréo 0–180°
AutoPan	Vitesse 0,05–10 Hz	Profondeur	Forme (3)	—
Phaser	Vitesse 0,05–5 Hz	Profondeur	Feedback 0–90 %	Étages 4/6/8/12
Flanger	Vitesse 0,05–5 Hz	Profondeur	Feedback ±95 %	Mix
Chorus	Vitesse 0,05–5 Hz	Profondeur	Largeur	Mix
Delay	Temps 20–1500 ms	Feedback 0–95 %	Mix	Stéréo / Ping-pong
Reverb	Taille	Amorti	Largeur	Mix
Master	Volume 0–150 %	Balance	Largeur 0–200 %	Limiteur

A.2 Coût de calcul et mémoire

Ordres de grandeur par trame stéréo (un échantillon gauche + un droit). Le budget disponible est d'environ 5 400 cycles processeur par trame à 240 MHz et 44,1 kHz.

Effet	Opérations principales par trame	Mémoire
Low/High/Band Pass	2 ou 4 biquads (10–20 multiplications)	—
EQ 3 bandes	6 biquads	—
Compressor	1 logarithme, 1 exponentielle	—
Distortion	2 courbes (exponentielle pour la fuzz), 2 anti-continu, 2 passe-bas	—
Bitcrusher	2 arrondis	—
Ring Mod	oscillateur en quadrature (ou 1 `lfoWave`), 2 multiplications	—
Tremolo	2 `lfoWave` (sinus), 2 lissages	—
AutoPan	1 `lfoWave`, 2 racines carrées	—
Phaser	$2N$ passe-tout ($N \leq 12$)	—
Flanger	2 lectures interpolées, 2 `softClip`	4 ko
Chorus	4 lectures interpolées	15 ko
Delay	2 lectures interpolées, 2 passe-bas, 2 `softClip`	576 ko
Reverb	16 peignes + 8 diffuseurs (accès PSRAM)	112 ko
Master	Mid/Side, limiteur, vumètres	—
Total PSRAM des effets		≈ 707 ko

La RAM interne (environ 320 ko, dont une grande part est prise par la pile Bluetooth) accueille le tampon d'entrée, le sprite de l'écran (40 ko) et les petites variables. La PSRAM de 4 Mo adressables accueille tous les grands tampons : il en reste plus de 3 Mo.

A.3 Références

— R. Bristow-Johnson, *Cookbook formulae for audio EQ biquad filter coefficients*. Formules de tous les biquads utilisés.

— U. Zölzer (dir.), *DAFX – Digital Audio Effects*, 2e éd., Wiley, 2011. Référence générale sur les effets audio numériques.

— J. O. Smith III, *Introduction to Digital Filters* et *Physical Audio Signal Processing*, CCRMA, Stanford (en ligne). Filtres, lignes à retard, peignes, passe-tout.

— M. R. Schroeder, « Natural sounding artificial reverberation », *J. Audio Eng. Soc.*, 1962.

— Jezar at Dreampoint, *Freeverb*, 2000 (domaine public). Algorithme de la reverb.

— D. Giannoulis, M. Massberg, J. D. Reiss, « Digital dynamic range compressor design – a tutorial and analysis », *J. Audio Eng. Soc.*, 2012. Genou doux et lissage dans le domaine des dB.

— J. Dattorro, « Effect design, part 1 & 2 », *J. Audio Eng. Soc.*, 1997. Chorus, flanger, réverbération.

— J. W. Cooley, J. W. Tukey, « An algorithm for the machine calculation of complex Fourier series », *Math. Comp.*, 1965. La FFT.

— P. Schatzmann, librairie *ESP32-A2DP* (GitHub). Réception Bluetooth A2DP et AVRCP.

— Espressif, *ESP32 Technical Reference Manual*. I2S, APLL, PSRAM.